



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
HIMALAYA COLLEGE OF ENGINEERING**

[CODE: ENCT 354]

A

FINAL PROJECT REPORT

ON

**ELECTRONICS ECOMMERCE PLATFORM WITH COSINE SIMILARITY
RECOMMENDATION AND MOBILENETV2 VISUAL SEARCH**

Submitted By

Aastha Mainali (HCE080BCT001)

Aayushma Ghale (HCE080BCT003)

Anjila Pandey (HCE080BCT007)

Anshu Regmi (HCE080BCT009)

A Final Project Report Submitted to Department of Electronics and Computer
Engineering in Fulfillment of the Requirement for Bachelor's Degree in Computer
Engineering

Department of Electronics and Computer Engineering

Lalitpur, Nepal

August 23, 2026

Electronics Ecommerce platform with Cosine Similarity Recommendation and MobileNetV2 Visual Search

BY

AASTHA MAINALI

AAYUSHMA GHALE

ANJILA PANDEY

ANSHU REGMI

A FINAL Report Submitted In Partial Fulfillment For The Requirement
Of The **Bachelor's Degree In Computer Engineering**

Under the supervision of

ISAN ADHIKARI

HIMALAYA COLLEGE OF ENGINEERING

Department of Electronics and Computer Engineering

Tribhuvan University

Lalitpur, Nepal

August 23, 2026

ACKNOWLEDGMENTS

We would like to express our sincere gratitude to the Institute of Engineering, Tribhuvan University, and Himalaya College of Engineering for providing us with the opportunity, academic environment, and resources to undertake and complete this project.

We are especially thankful to the Department of Electronics and Computer Engineering, Himalaya College of Engineering, for its continuous support and guidance throughout the course of this project. We would like to express our heartfelt gratitude to our project supervisor, **Er. Isan Adhikari**, for his continuous guidance, valuable suggestions, constructive feedback, and encouragement throughout the development of this project. His support and expertise have been invaluable in successfully completing this work.

We would also like to extend our sincere appreciation to the Head of Department, Project Coordinator, **Er. Hasina Shakya**, and all the respected faculty members of the Department of Electronics and Computer Engineering for their support, coordination, guidance, and encouragement during different stages of the project.

We are grateful to our classmates and friends for their cooperation, suggestions, and support throughout the project. We would also like to acknowledge everyone who has directly or indirectly contributed to the successful completion of this work.

ABSTRACT

The project “Electronics Marketplace with Cosine Similarity Recommendation and MobileNetV2 Visual Search” focuses on developing an AI-enabled e-commerce platform for discovering, comparing, and purchasing smartphones and laptops across multiple sellers. Product data spanning category, brand, specifications, and description was processed through data cleaning, feature extraction, and feature engineering to obtain TF-IDF text vectors, one-hot encoded categorical attributes, and min-max scaled numerical specifications. A content-based recommendation engine was implemented using weighted cosine similarity over these combined feature vectors, with weights tuned via grid search and verified through sensitivity analysis, while a semantic search module using Sentence-Transformer embeddings and a visual search module using a pretrained MobileNetV2 convolutional neural network were implemented for alternative product discovery. All three components were evaluated using Precision@K, Recall@K, Hit Rate@K, MRR, and NDCG@K, and their results were benchmarked against a random-recommendation baseline to assess effectiveness. The completed system integrates these intelligent discovery features with core e-commerce functionalities, including role-based authentication, seller management, cart and order processing, and online payment, providing customers with a centralized platform for exploring and purchasing electronics while enabling sellers and administrators to manage the marketplace through dedicated interfaces.

Keywords: *E-Commerce, Electronics Marketplace, Content-Based Recommendation, Cosine Similarity, TF-IDF, Semantic Search, Visual Search, MobileNetV2*

TABLE OF CONTENTS

ACKNOWLEDGMENTS	i
ABSTRACT	ii
LIST OF FIGURES	vi
LIST OF TABLES	viii
LIST OF ABBREVIATIONS	ix
1 INTRODUCTION	1
1.1 Background Information	1
1.2 Content-Based Recommendation	2
1.3 Semantic Search	2
1.4 Visual Search	2
1.5 Problem Statement	3
1.6 Objectives	3
1.7 Scope and Applications	4
2 LITERATURE REVIEW	5
2.1 E-Commerce and Product Discovery	5
2.2 Content-Based Recommendation Systems	6
2.3 Semantic Product Search	6
2.4 Visual Product Search	7
2.5 Identified Research Gaps	7
3 REQUIREMENT ANALYSIS AND SYSTEM DESIGN	9
3.1 Requirement Analysis	9
3.1.1 Functional Requirements	9
3.1.2 Non-Functional Requirements	11
3.2 System Design	11

3.2.1	ER Diagram	11
3.2.2	Data Flow Diagram	13
3.3	Activity Diagram	14
4	METHODOLOGY	16
4.1	Basic System Architecture	16
4.1.1	System Flowcharts	17
4.1.2	Algorithm: Cosine Similarity for Product Recommendation . . .	22
4.2	Incremental Development Model	26
4.3	Dataset Description	27
4.3.1	Dataset	27
4.3.2	Visual Search Dataset	28
4.4	Tools Used	30
5	RESULTS AND ANALYSIS	31
5.1	Backend Development (Django REST Framework)	31
5.2	Recommendation Engine Development and Evaluation	32
5.3	Visual Search Development and Evaluation	35
5.4	Semantic Search Development and Evaluation	37
5.5	Frontend Development (React with Tailwind CSS)	39
5.6	Version Control and Team Collaboration	40
5.7	Verification	40
5.8	Validation	41
6	FUTURE ENHANCEMENTS	42
7	CONCLUSION	43
	REFERENCES	44
	APPENDICES	45
7.1	APPENDIX A: Technical Specifications	45
7.1.1	Software Requirements	45
7.1.2	Hardware Requirements	45
7.2	APPENDIX B: Dataset Sample	46

7.3	APPENDIX C: Recommendation Engine Terminal Output	49
7.4	APPENDIX D: User Interface Screenshots	50
7.4.1	Customer Interfaces	50
7.4.2	Seller Interfaces	53
7.4.3	Administrator Interfaces	54

LIST OF FIGURES

3.1	Use Case Diagram of the Smartphone and Laptop Marketplace	10
3.2	Entity Relationship Diagram	12
3.3	DFD Level 0	13
3.4	Activity Diagram - Login, Role Assignment & Product Discovery . . .	14
3.5	Activity Diagram - Cart & Order placement & Payment	15
4.1	System Architecture Diagram	16
4.2	Recommendation Engine Flow Diagram	17
4.3	Visual (Image) Search Pipeline Diagram	19
4.4	Semantic Search Pipeline Diagram	21
4.5	Incremental Development Model Diagram	26
5.1	Weight Sensitivity Analysis – Text Weight	33
5.2	Weight Sensitivity Analysis – Category Weight	33
5.3	Weight Sensitivity Analysis – Spec Weight	34
5.4	Recommendation Quality: Model vs Random Baseline	34
5.5	Recall@5 versus its Theoretical Ceiling	35
5.6	Visual Search Quality: Model vs Category-Random vs Pure-Random Baseline	36
5.7	Visual Search Recall@5 versus its Theoretical Ceiling	37
5.8	Semantic Search Quality: Model vs Category-Random vs Pure-Random Baseline	38
5.9	Semantic Search Recall@5 versus its Theoretical Ceiling	39
7.1	Sample of the 10,000-item combined product dataset used by the rec- ommendation engine.	46
7.2	Sample of the 10,000-item combined product dataset used by the rec- ommendation engine.	47

7.3	Sample of the Amazon Electronics Products 10k Items – 2023 dataset used for visual search.	48
7.4	Terminal Output: Top-5 Recommendation Demonstration	49
7.5	Terminal Output: Top-5 Recommendation Demonstration	49
7.6	Sign In Interface	50
7.7	Product Detail Interface	50
7.8	Visual Search Interface	51
7.9	Semantic Search Interface	51
7.10	Checkout Page	52
7.11	Khalti Payment Portal	53
7.12	Seller Dashboard Interface	53
7.13	Order Management Dashboard (Paid/Unpaid)	54

LIST OF TABLES

4.1	Description of the combined dataset columns.	28
4.2	Development Tools and Technologies	30
7.1	Software Requirements of the Proposed System	45

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CBF	Content-Based Filtering
CRUD	Create, Read, Update, Delete
DRF	Django REST Framework
HTTP	HyperText Transfer Protocol
MRR	Mean Reciprocal Rank
MVC	Model-View-Controller
MySQL	My Structured Query Language
NDCG	Normalized Discounted Cumulative Gain
OpenCV	Open Source Computer Vision Library
ORM	Object-Relational Mapping
REST	Representational State Transfer
SPA	Single-Page Application
TF-IDF	Term Frequency–Inverse Document Frequency

CHAPTER 1:

INTRODUCTION

1.1 Background Information

The rapid growth of e-commerce has changed the way consumers discover, compare, and purchase electronic products. Smartphones and laptops contain numerous specifications such as processor, RAM, storage, camera, display, battery, and price, making product selection difficult when a large number of similar products are available. Conventional e-commerce platforms generally depend on keyword-based search and manual comparison, which may not provide effective product discovery when customers do not know the exact product name or model.

GadgetHub is an electronics e-commerce platform designed to improve product discovery and purchasing of smartphones and laptops. The platform supports customers, sellers, and administrators through role-based access control. Customers can browse products, search the catalog, receive product recommendations, perform visual searches, manage favorites and carts, make payments, and view their orders. Sellers can register and manage their products, while administrators can manage users, sellers, products, and other platform activities. Thus, the system provides multi-vendor support while maintaining a centralized product catalog and administration.

The primary product discovery component of GadgetHub is a content-based recommendation system based on cosine similarity. Product information is represented using textual, categorical, and numerical attributes. TF-IDF is used to represent textual information, one-hot encoding is applied to categorical attributes, and Min-Max normalization is used for numerical specifications. These feature representations are combined to calculate product similarity and recommend products related to the currently viewed item.

GadgetHub also provides semantic product search and visual search to provide alternative methods of product discovery. The visual search system uses OpenCV for image preprocessing and a pretrained MobileNetV2 convolutional neural network for extracting image feature embeddings. The extracted features are compared with catalog image

features using cosine similarity to retrieve visually similar products. The platform is implemented using React.js, Django REST Framework, and MySQL, with Khalti integrated for online payment processing.

1.2 Content-Based Recommendation

Content-based recommendation is a technique that recommends items by comparing their characteristics with those of an item in which the user is currently interested. Unlike collaborative filtering, it does not require a large amount of historical interaction or purchase data to identify similar products.

In GadgetHub, product attributes are transformed into numerical feature representations and combined into a product feature vector. Textual attributes are represented using TF-IDF, categorical attributes using one-hot encoding, and numerical specifications using Min-Max normalization. Cosine similarity is then calculated between product vectors, and products with higher similarity scores are considered relevant recommendations. This approach allows the system to recommend alternatives based on actual product characteristics rather than relying only on brand or category matching.

1.3 Semantic Search

Semantic search allows users to search for products using descriptive queries rather than relying only on exact product names. In an electronics catalog, customers may search using terms related to product categories, brands, specifications, or desired features. GadgetHub provides a semantic search interface to improve product discovery by matching user queries with relevant product information in the catalog.

1.4 Visual Search

Visual search enables product discovery using an image instead of a textual query. This is useful when a customer recognizes the appearance of a device but does not know its exact product name or model. GadgetHub implements visual search using OpenCV for image preprocessing and a pretrained MobileNetV2 model for extracting visual feature embeddings. The extracted features are compared with the image representations of catalog products using cosine similarity to retrieve visually similar products.

1.5 Problem Statement

The major problems addressed by GadgetHub are:

- **Limited product discovery:** Conventional keyword-based search may fail to provide suitable alternatives when customers do not know the exact product name or model.
- **Difficulty in comparing electronic products:** Smartphones and laptops contain numerous technical specifications, making manual comparison across a large catalog time-consuming.
- **Limited recommendation based on product characteristics:** Customers may not be presented with relevant alternatives based on the specifications of the product they are viewing.
- **Lack of image-based product discovery:** Customers may know the appearance of a product without knowing its exact name, making conventional text-based search insufficient.
- **Multi-vendor management requirements:** A platform involving multiple sellers requires structured seller registration, product listing, customer–seller relationships, and administrative management.

1.6 Objectives

The main objectives of this project are:

1. To develop a secure electronics e-commerce platform supporting customer, seller, and administrator roles, with product browsing, seller management, product listing, favorites, cart, checkout, online payment, order management, and administrative functionalities.
2. To implement content-based product recommendation using cosine similarity, semantic product search, and visual product search using MobileNetV2 to help customers discover relevant and similar electronics products.

1.7 Scope and Applications

There are various areas where the proposed system can be implemented. Some of its potential applications are mentioned below:

- It can be used by online electronics retailers for selling smartphones, laptops, and other consumer electronic products.
- It can be adopted by local electronics stores and distributors to establish an online presence and reach customers through a centralized e-commerce platform.
- It can be used as a multi-vendor platform where multiple electronics businesses can offer their products through a common online marketplace.
- It can support electronics marketplaces and retail platforms that require improved product discovery and recommendation for large product catalogs.
- It can be applied in product comparison and electronics shopping platforms where customers need to discover and evaluate products based on their specifications.

CHAPTER 2:

LITERATURE REVIEW

This chapter reviews previous research related to electronics e-commerce, content-based recommendation, semantic product search, and visual product retrieval. The reviewed studies provide the theoretical foundation for the major components implemented in GadgetHub.

2.1 E-Commerce and Product Discovery

Electronic commerce platforms provide online environments for product discovery, comparison, and purchasing. The increasing number and variety of products available through online platforms has made effective product search an important component of e-commerce systems. Research on product search has shown that product catalogs contain heterogeneous attributes such as categories, brands, and product specifications, making product retrieval more complex than conventional document search [1].

Product information can be represented using multiple attributes to support search and comparison. Kim et al. proposed a product information meta-search framework that considers multiple product attributes for electronic commerce, demonstrating the importance of structured product information in product search and comparison [2]. Such approaches are relevant to electronics platforms where products contain several specifications that customers may use when searching for suitable products.

The proposed GadgetHub platform applies this concept to smartphones and laptops by maintaining structured product information including category, brand, price, processor, memory, storage, display, camera, battery, and other specifications. The platform combines conventional e-commerce functionality with recommendation and search techniques to improve product discovery.

2.2 Content-Based Recommendation Systems

Recommendation systems are widely used to help users discover relevant items from large collections of products. Recommendation approaches are commonly classified into collaborative filtering, content-based filtering, and hybrid methods. Content-based recommendation focuses on the characteristics of items and recommends products that are similar to the item of interest [3].

Lops, de Gemmis, and Semeraro reviewed content-based recommender systems and described the process of representing item characteristics and matching these representations to identify relevant items [3]. This approach is suitable for applications where recommendations can be generated from available item attributes without depending entirely on large amounts of user interaction data.

The vector space model introduced by Salton, Wong, and Yang represents information objects as vectors and enables similarity-based comparison between them [4]. Cosine similarity can therefore be used to determine the degree of similarity between product representations. In text retrieval, Salton and Buckley further investigated term-weighting methods for representing the importance of terms in documents and queries [5].

GadgetHub applies content-based recommendation using product attributes. Textual product information is represented using TF-IDF, categorical attributes are encoded using one-hot encoding, and numerical specifications are normalized before being combined into product feature representations. Cosine similarity is then used to identify products that are similar to the currently viewed product. This provides recommendations based on product characteristics rather than requiring collaborative user-rating data.

2.3 Semantic Product Search

Keyword-based product search may be limited when a customer's query does not exactly match the terms used in a product catalog. Research in e-commerce information retrieval has therefore investigated semantic matching between customer queries and product information.

Nigam et al. studied semantic matching for product search and identified limitations of purely lexical matching, including difficulties with synonyms, morphological variations,

and spelling differences [6]. Similarly, Choi et al. investigated semantic product search over structured e-commerce catalogs and emphasized the importance of representing multiple product fields when matching customer queries with products [1].

GadgetHub provides a product search interface that uses product information and descriptive search terms to retrieve relevant electronics from the catalog. The search component is intended to reduce dependence on exact product names and allow customers to discover products using category, brand, and specification-related terms.

2.4 Visual Product Search

Visual search provides an alternative method of product discovery by using an image as the search input. Image retrieval research has demonstrated the use of learned visual representations to identify visually related products from large image collections. For example, Liu et al. demonstrated the use of deep visual representations for image-based product retrieval in the DeepFashion system [7].

MobileNetV2, proposed by Sandler et al., introduced an efficient convolutional neural network architecture using inverted residuals and linear bottlenecks, providing a lightweight model suitable for computer vision applications [8]. Its learned representations can be used as feature embeddings for comparing images.

GadgetHub implements visual product search by preprocessing uploaded images, extracting visual features using a pretrained MobileNetV2 model, and comparing the resulting feature representations with catalog image features using cosine similarity. The retrieved products provide visually similar alternatives to the uploaded product image.

2.5 Identified Research Gaps

The reviewed literature demonstrates the effectiveness of content-based recommendation, semantic product search, and image-based retrieval as individual techniques. However, two gaps can be identified specifically within the scope of the reviewed studies:

- (i) **Separate feature representations across techniques:** The reviewed studies generally address content-based recommendation [3, 4, 5], semantic search [6, 1], and

visual search [7, 8] using feature representations suited to a single technique at a time (e.g., term-weighted text vectors for recommendation, or learned visual embeddings for image retrieval). None of the reviewed studies address how textual, categorical, and numerical product attributes, together with visual features, can be represented under a shared product schema so that recommendation, search, and visual retrieval can operate consistently over the same catalog.

- (ii) **Independent evaluation rather than integrated operation:** The reviewed studies on content-based recommendation [3, 4, 5], semantic product search [6, 1], and visual product search [7, 8] each evaluate their respective techniques as standalone components. None of the reviewed studies demonstrate these three techniques implemented and operating together within a single working system, where outputs from one component (e.g., product attributes used for recommendation) must remain consistent with the catalog used by the others (e.g., search and visual retrieval).

The GadgetHub system addresses these two gaps by defining a common product schema for smartphones and laptops that supports content-based recommendation, semantic product search, and MobileNetV2-based visual search, and by implementing and evaluating all three components together within a single integrated platform rather than as isolated modules.

CHAPTER 3:

REQUIREMENT ANALYSIS AND SYSTEM DESIGN

3.1 Requirement Analysis

3.1.1 Functional Requirements

a) Customer Module:

- Secure registration and login.
- Semantic keyword search with faceted filtering (price, RAM, storage, brand, OS).
- Visual image search by photo upload.
- Real-time behavior tracking (views, browsing sequence, cart activity).
- Personalized homepage and item-detail recommendation carousels.
- Persistent cart and wishlist with stock checking.
- Checkout with order placement and payment processing.

b) Seller Module:

- Registration and profile management.
- Seller dashboard displaying total orders and order status overview.
- Product catalog management (listing, editing, images, price in NPR, stock).
- View of placed orders for products listed by the seller.

c) Administrator Module:

- Management of users, products, brands, and categories.
- Admin dashboard for marketplace-wide monitoring.
- Seller verification, where the administrator reviews pending seller registrations and approves or rejects them from the admin dashboard.
- Order management and product moderation across the marketplace.

3.1.1.1 Use Case Diagram

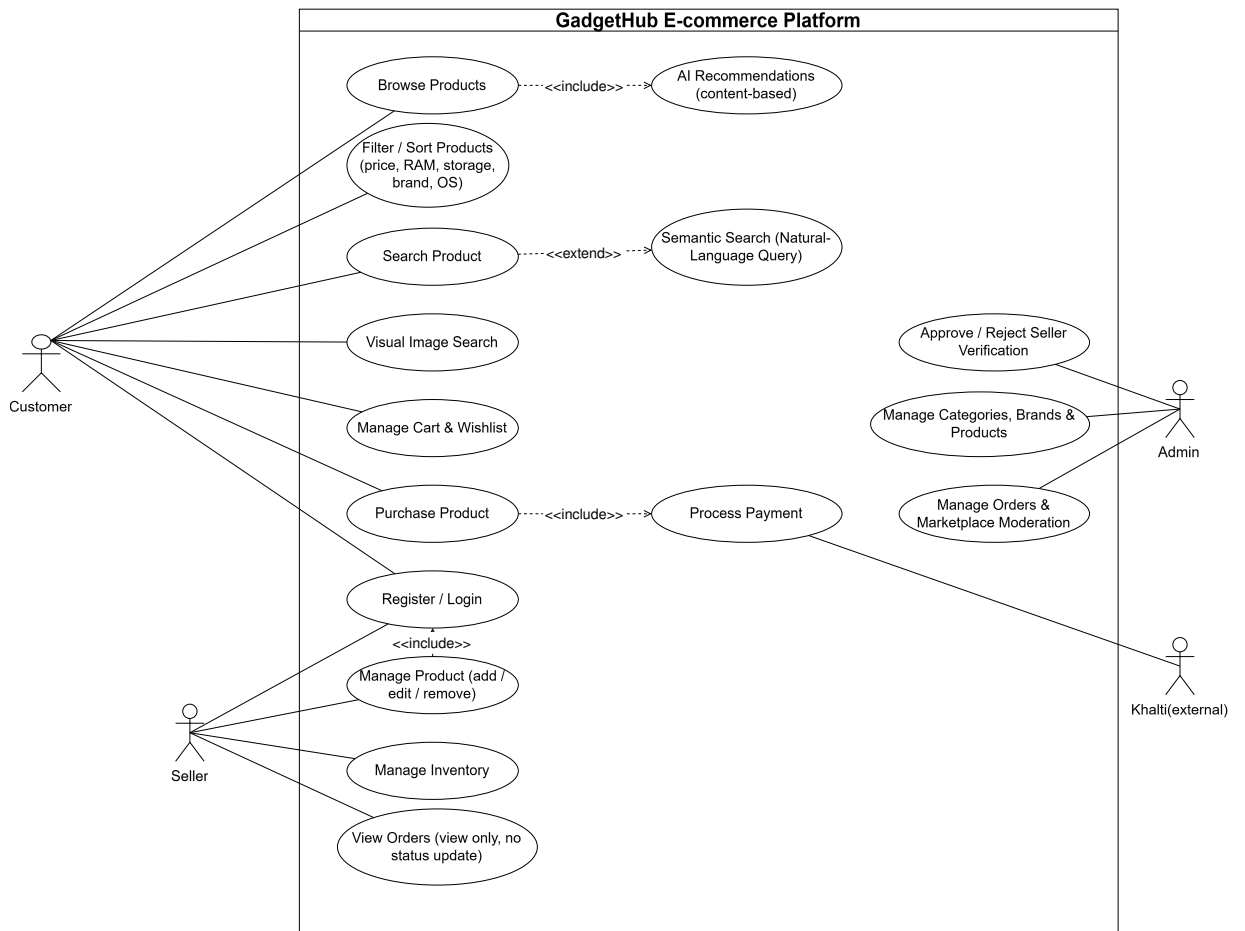


Figure 3.1: Use Case Diagram of the Smartphone and Laptop Marketplace

The diagram isolates how each actor interacts with the system: customers use semantic search, filtering, tracking-driven recommendations, and visual search; sellers manage inventory; administrators moderate the marketplace.

- **Customer:** Registers/logs in; browses products with faceted filtering and sorting (price, RAM, storage, brand, OS); searches products using semantic keyword search; searches using uploaded product images; manages cart/wishlist; triggers behavioral tracking for personalized recommendations; completes checkout and payment via Khalti.
- **Seller:** Manages product listings, pricing, and stock; views placed orders and total order counts; does not currently manage order status or revenue analytics.
- **Administrator:** Approves or rejects seller verification requests from the admin dashboard; manages categories, brands, and products; oversees orders and

marketplace-wide moderation.

3.1.2 Non-Functional Requirements

- **Performance:** API responses and recommendation/similarity computations return within an acceptable response time for interactive use.
- **Scalability:** The modular Django backend architecture accommodates growth in catalog size and concurrent users.
- **Security:** JWT-based authentication, Django’s built-in password hashing, and role-based access control (RBAC) are enforced at the backend to separate customer, seller, and admin permissions.
- **Usability:** A responsive React single-page application (SPA) interface is provided; semantic search allows queries without requiring exact product model numbers.
- **Reliability:** Transactional data integrity is supported by MySQL and Django’s ORM to maintain consistent order and inventory records.
- **Maintainability:** The codebase follows a modular, separation-of-concerns structure with Django ORM-managed database access (MySQL) for portability and ease of future modification.

3.2 System Design

3.2.1 ER Diagram

The ER diagram shows relationships among the system’s primary entities: User, Seller, Product, Brand, Category, Product Specification, Order, Order Item, Cart, and Wishlist. A User places many Orders and may register as a Seller; a Seller lists many Products; a Product belongs to a Brand and a Category; an Order contains many Order Items; a User saves items to a Wishlist.

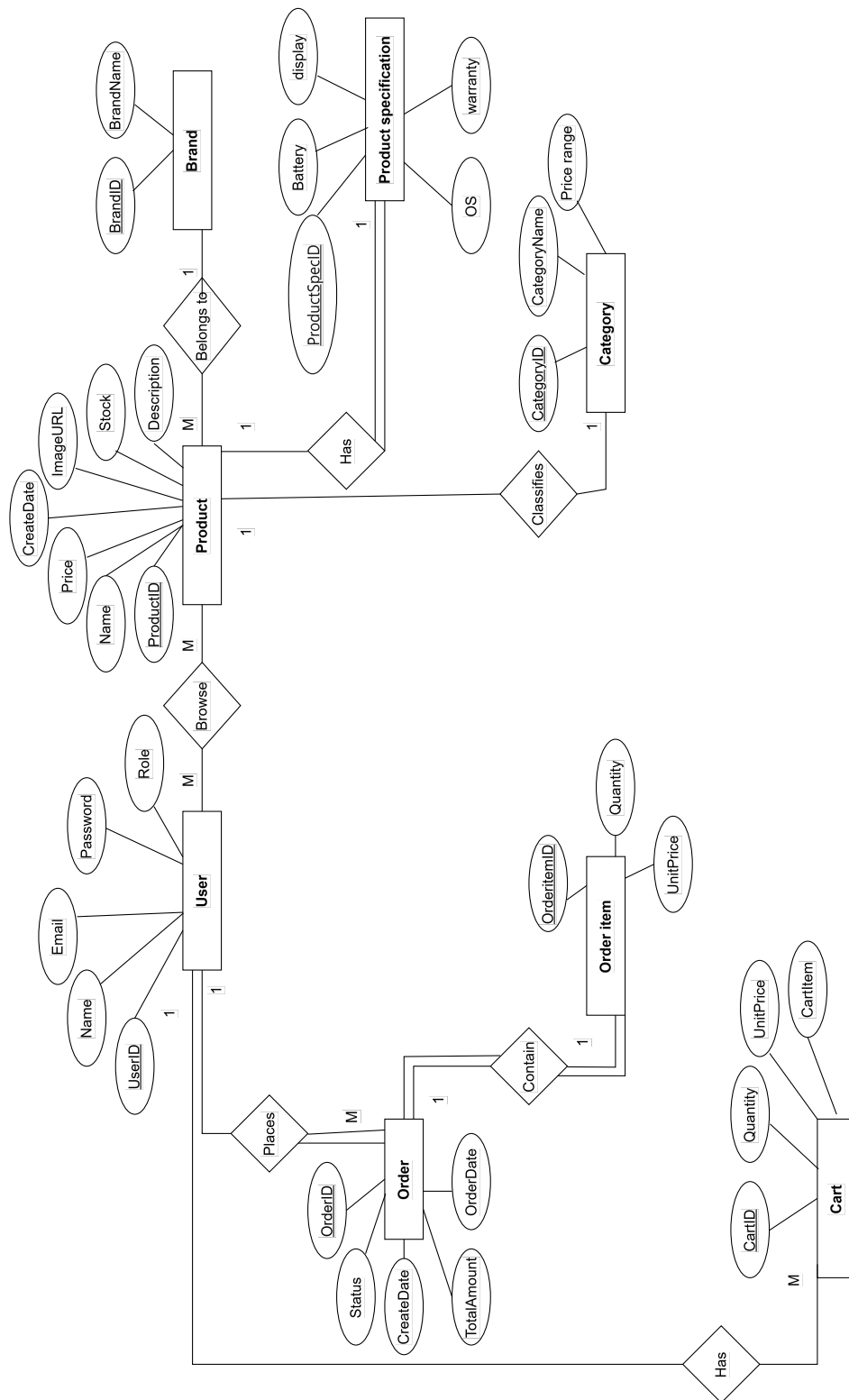


Figure 3.2: Entity Relationship Diagram

3.2.2 Data Flow Diagram

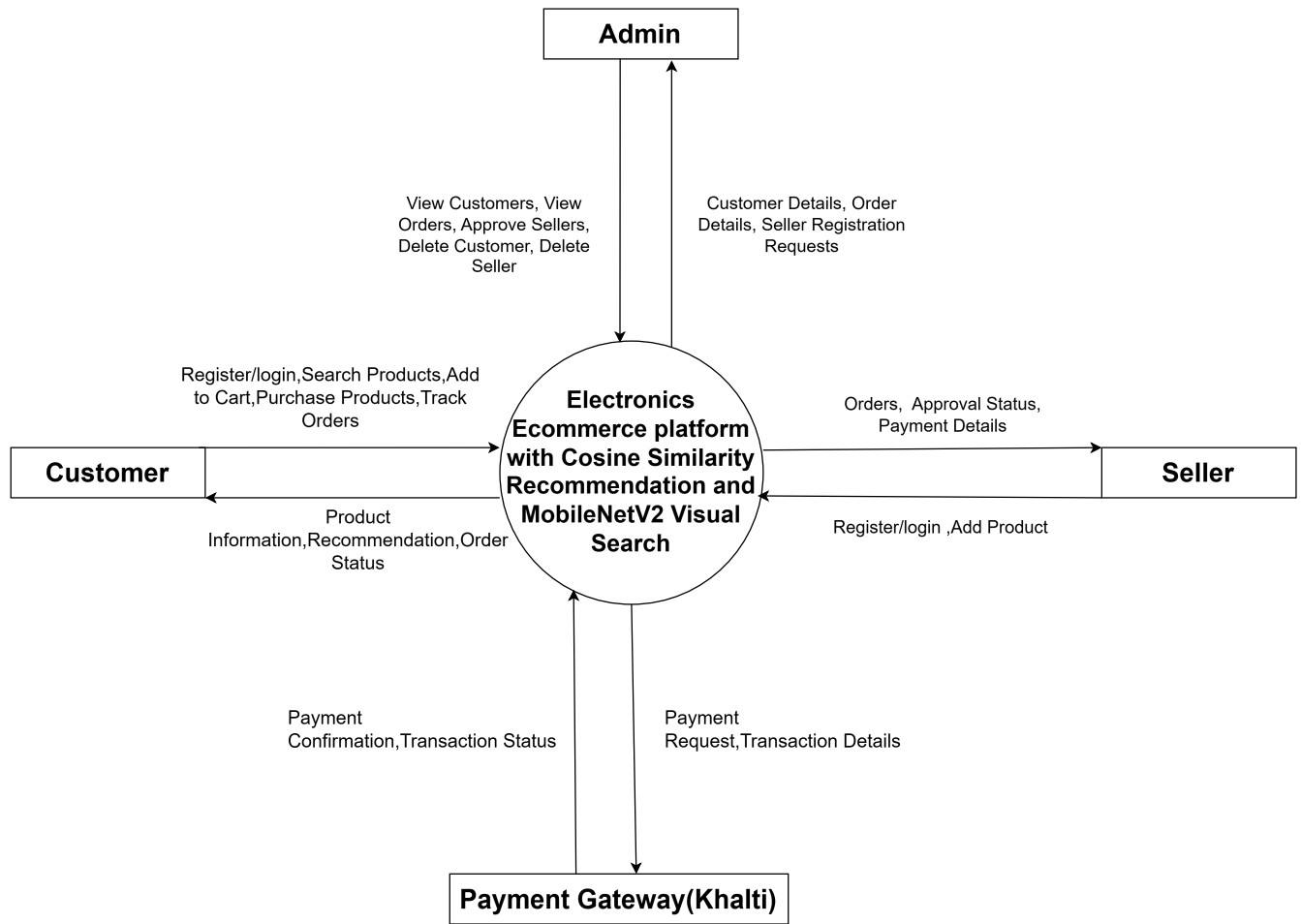


Figure 3.3: DFD Level 0

3.3 Activity Diagram

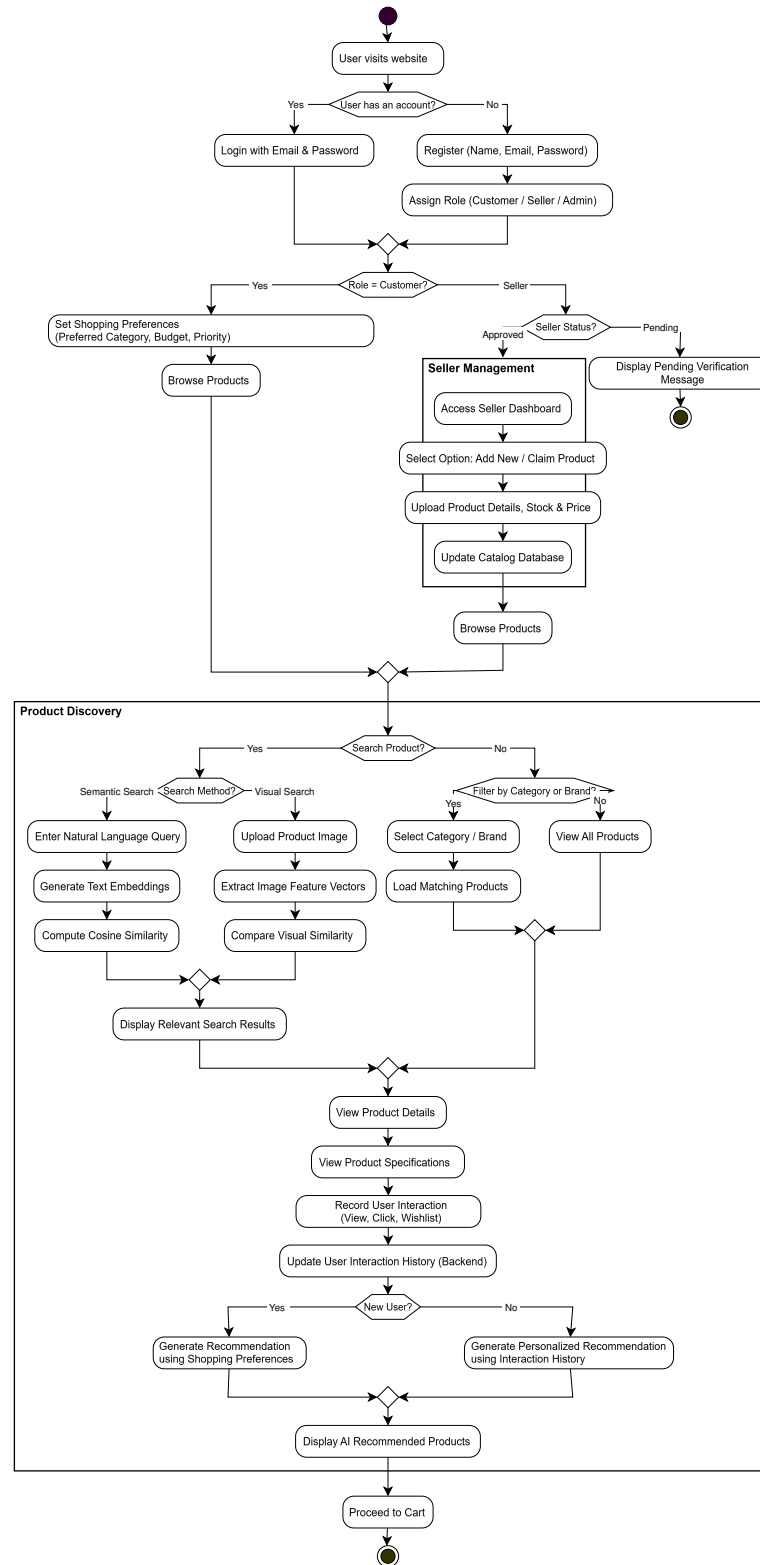


Figure 3.4: Activity Diagram - Login, Role Assignment & Product Discovery

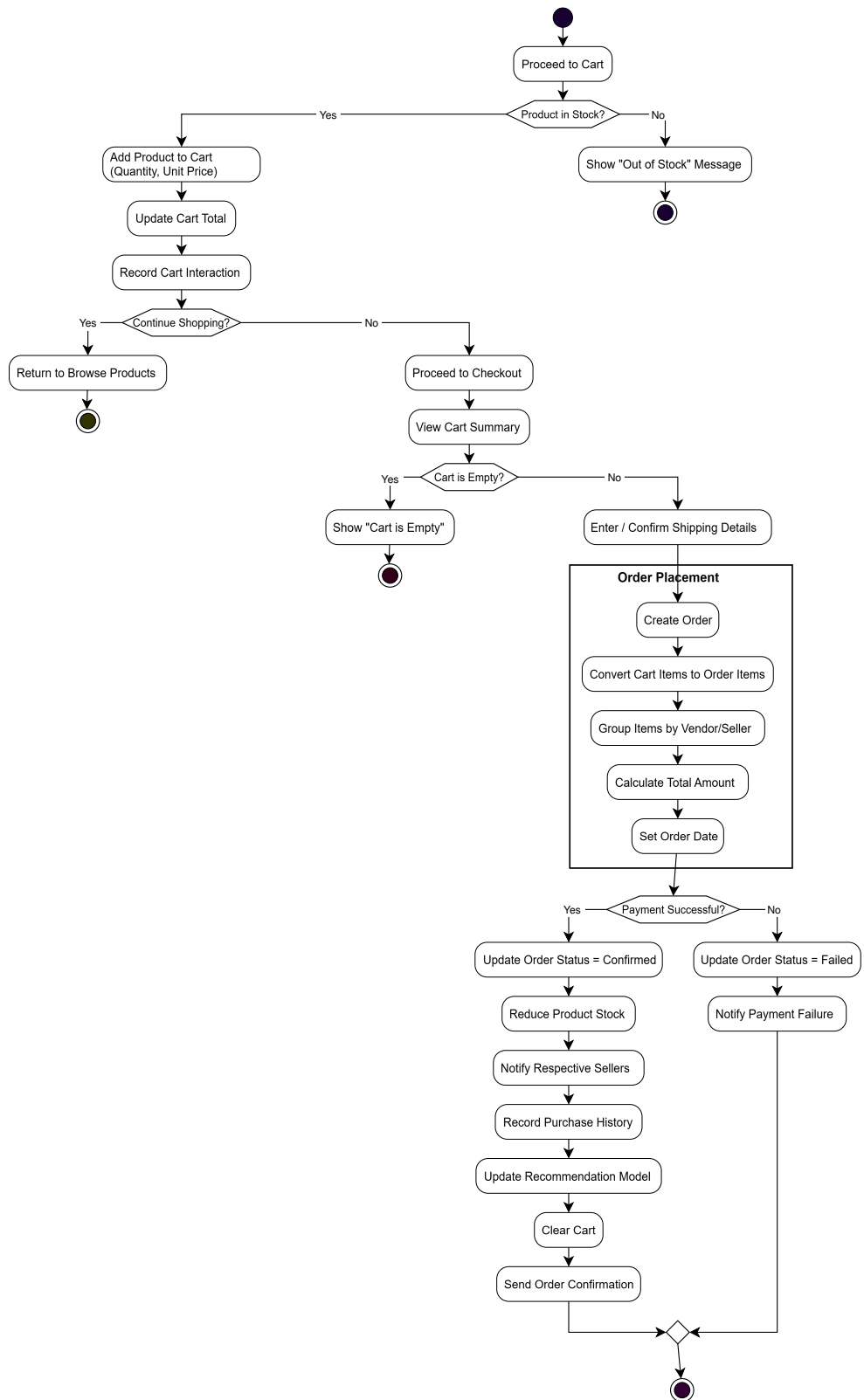


Figure 3.5: Activity Diagram - Cart & Order placement & Payment

CHAPTER 4:

METHODOLOGY

4.1 Basic System Architecture

The system follows a three-tier client-server architecture. The presentation tier is a React.js SPA communicating with the backend through RESTful HTTP APIs. The application tier, built with Django REST Framework, handles business logic, authentication, recommendation computation, and third-party API integration. The data tier is a MySQL database accessed through Django's ORM.

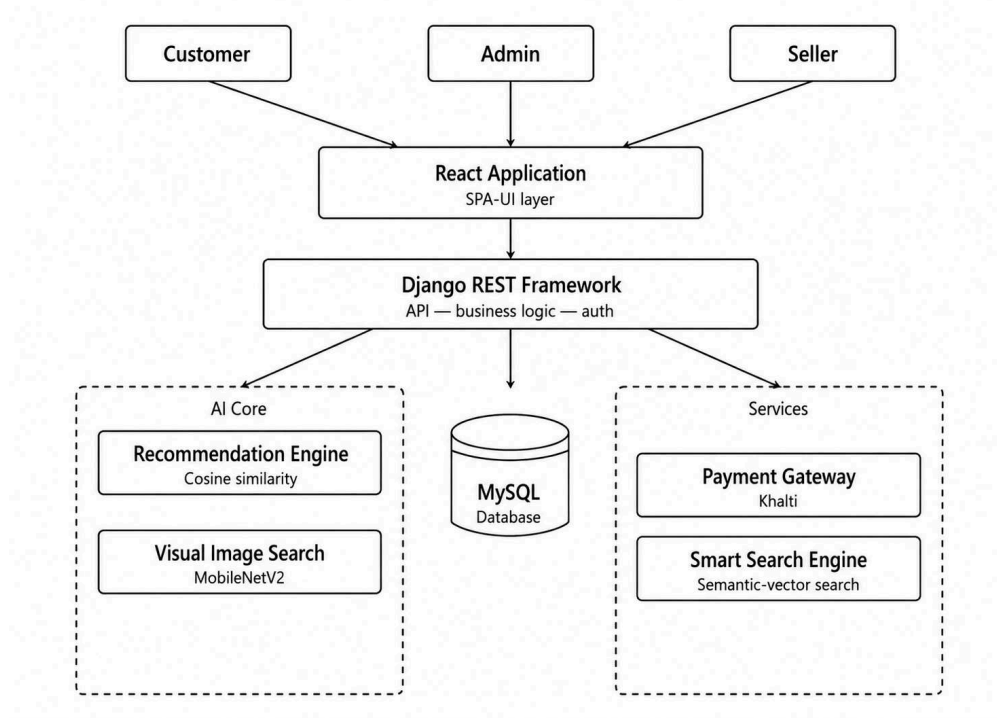


Figure 4.1: System Architecture Diagram

The AI Core, comprising the Recommendation Engine, Semantic Search Engine, and Visual Search Engine, sits in the application tier as service modules. The Recommendation Engine computes weighted cosine similarity over product attribute vectors using Scikit-learn, invoked by the DRF views serving the product detail and homepage endpoints. External services, Khalti for payments and an email provider for notifications, interface with the application tier over HTTPS.

4.1.1 System Flowcharts

Recommendation Engine Flow

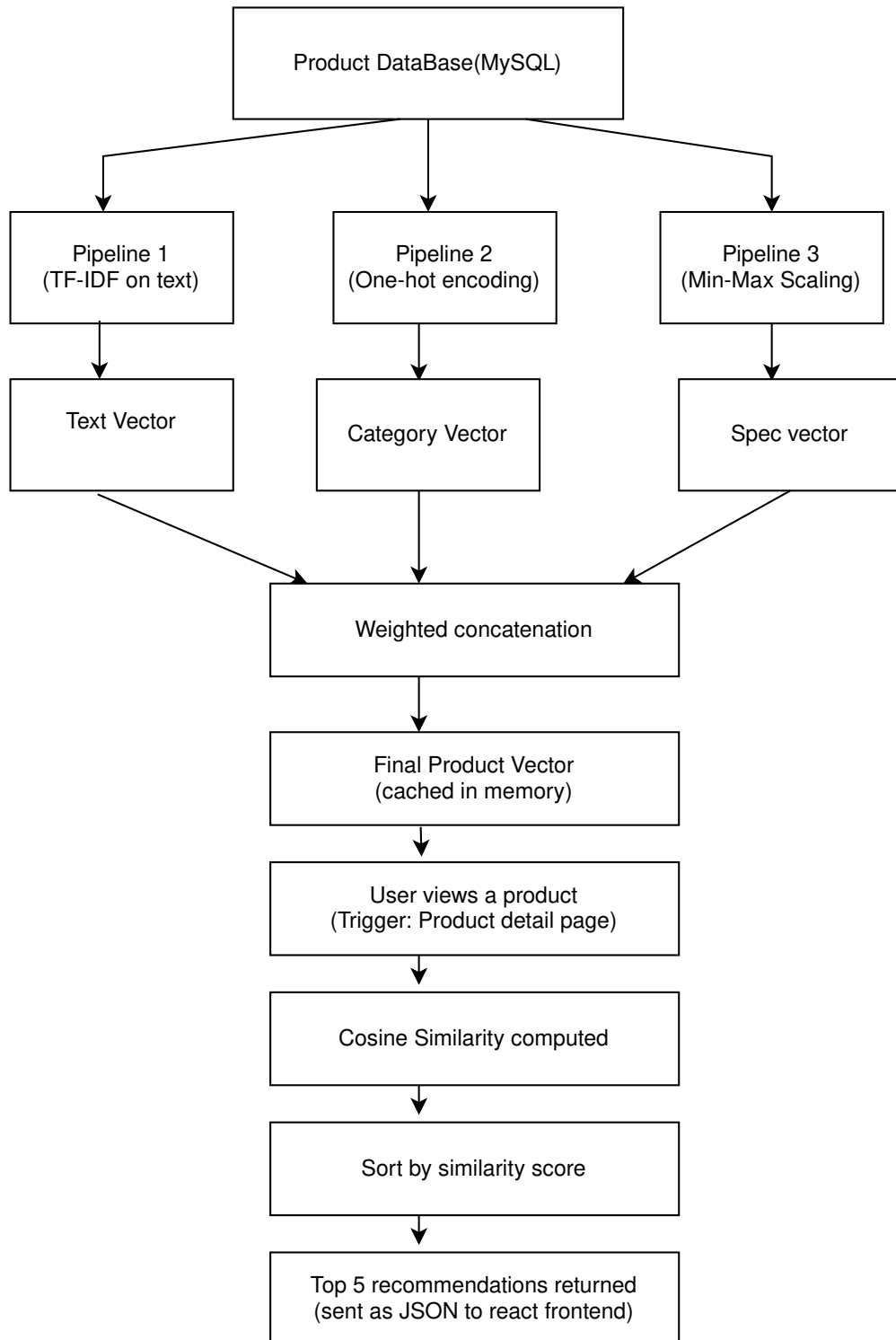


Figure 4.2: Recommendation Engine Flow Diagram

The engine begins when a user views, searches, or purchases a product. Category, brand, specifications, and description are pulled from the catalog; features are extracted using TF-IDF (text), one-hot encoding (categorical), and min-max scaling (numerical), then combined via weighted concatenation into a single product vector. Cosine similarity is computed between products, and the top-ranked results are shown as personalized recommendations.

VISUAL SEARCH FLOW

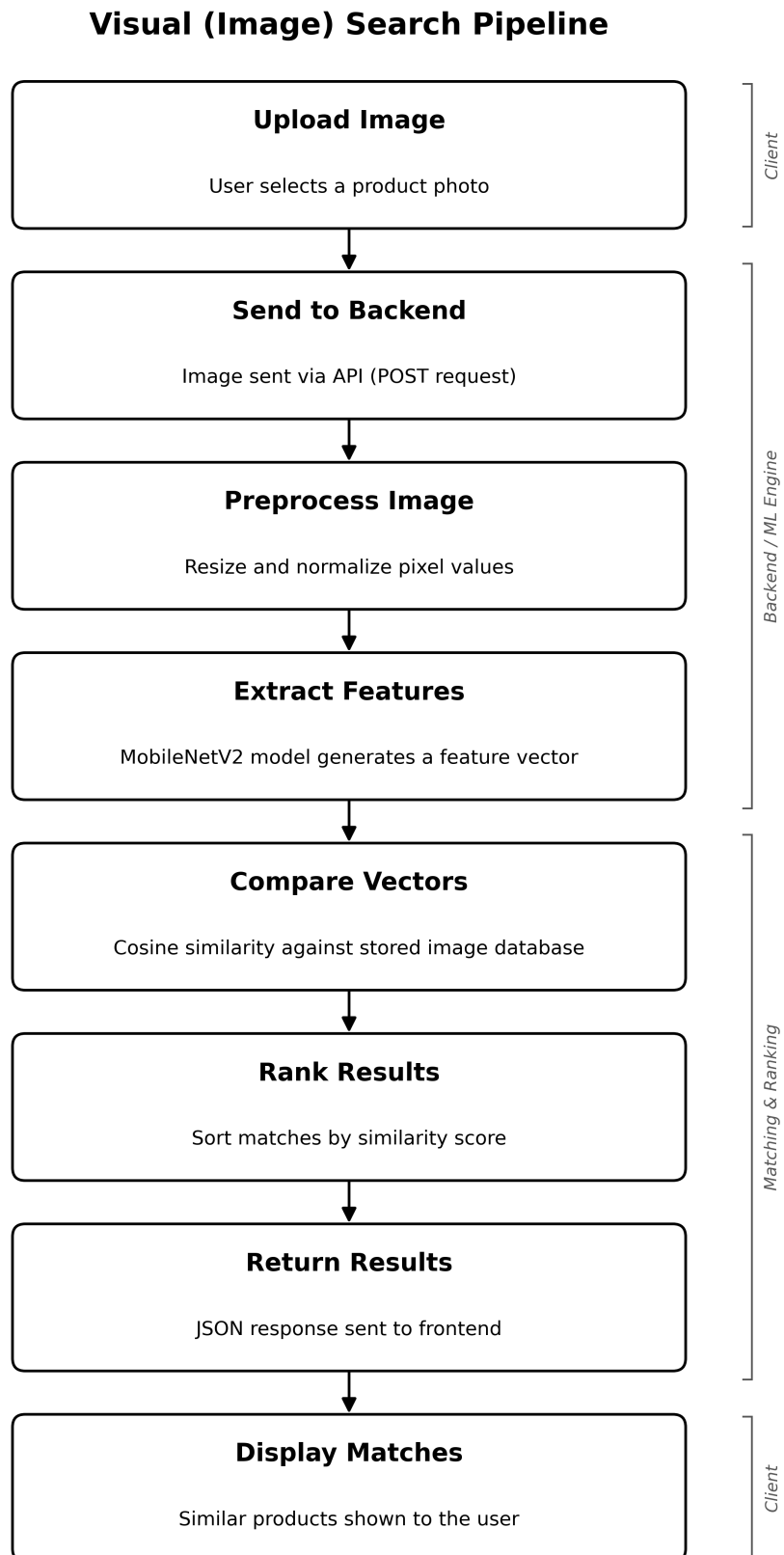


Figure 4.3: Visual (Image) Search Pipeline Diagram

The pipeline begins when a user uploads a product photo from the client interface. The image is sent to the backend via a POST request, where it is preprocessed through resizing and pixel normalization to match the input requirements of the feature extraction model. A pretrained MobileNetV2 model then generates a dense feature vector representing the image's visual characteristics. This vector is compared against the feature vectors of stored catalog images using cosine similarity, and the resulting matches are ranked by similarity score. The top-ranked results are returned as a JSON response to the frontend and displayed to the user as visually similar product matches.

SEMANTIC SEARCH FLOW

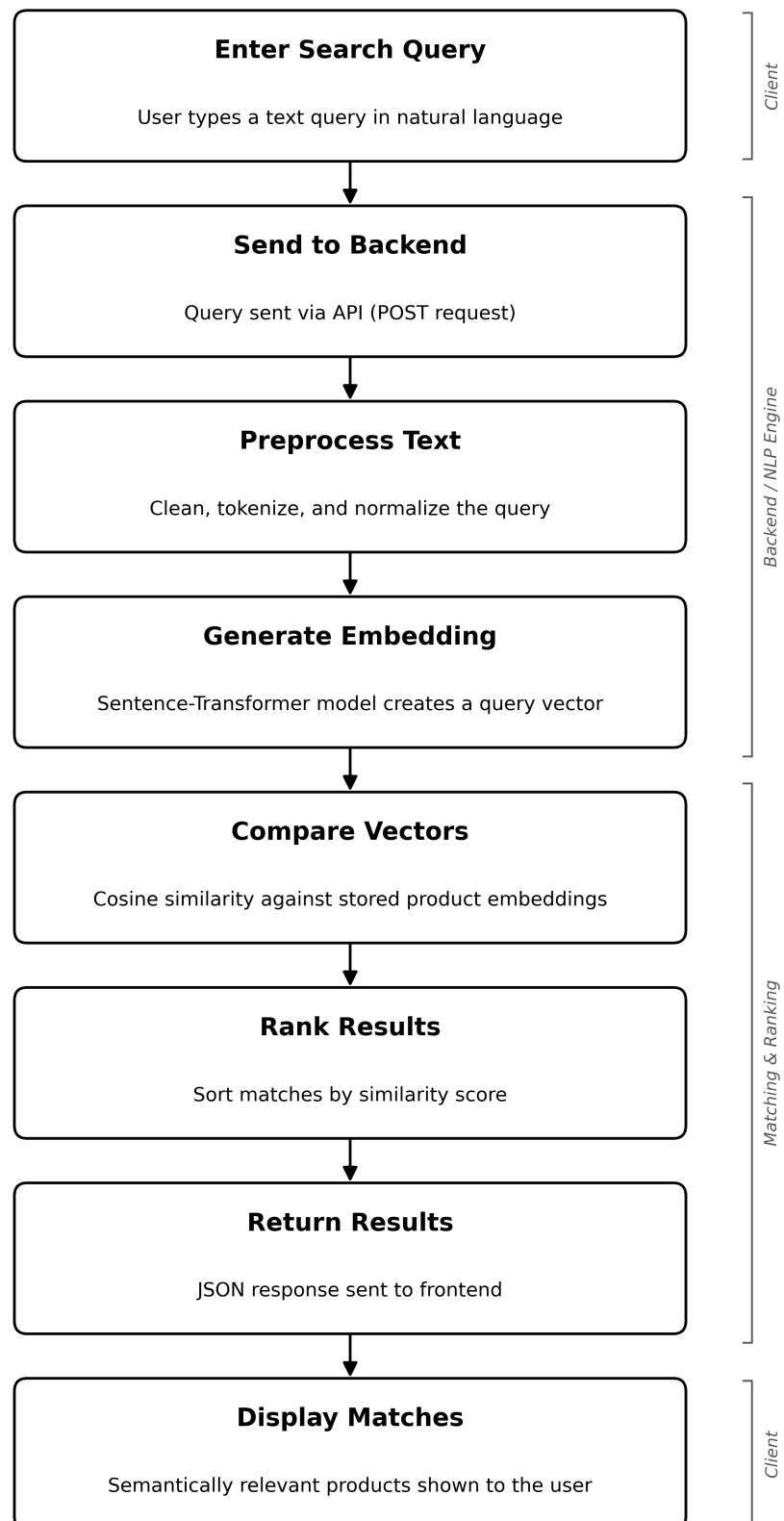


Figure 4.4: Semantic Search Pipeline Diagram

The pipeline begins when a user enters a natural language search query on the client interface. The query is sent to the backend via a POST request, where it is cleaned, tokenized, and normalized. A pretrained Sentence-Transformer model then generates a dense embedding vector representing the semantic meaning of the query. This vector is compared against the precomputed embeddings of catalog products using cosine similarity, and the resulting matches are ranked by similarity score. The top-ranked results are returned as a JSON response to the frontend and displayed to the user as semantically relevant product matches.

4.1.2 Algorithm: Cosine Similarity for Product Recommendation

Purpose and Application

Cosine similarity measures how alike two products are based on their attribute vectors, ranking the catalog by decreasing similarity to the product a user is currently viewing. The engine retrieves the viewed product’s vector, computes similarity against every other product, and returns the top-5 most similar products on the product detail page and homepage.

Why Cosine Similarity

Three reasons drove this choice. **Cold-start robustness:** unlike collaborative filtering, it requires no prior user history, making it suitable for a newly launched platform. **Efficiency:** pairwise similarity over TF-IDF, one-hot, and numerical matrices via Scikit-learn needs no GPU or training pipeline. **Interpretability:** the score directly reflects similarity in category, brand, price, and specifications, making recommendations auditable.

Feature Vector Construction

Each product is represented as $\mathbf{v} \in \mathbb{R}^n$, built from three attribute blocks combined by weighted concatenation.

Text block (TF-IDF). For term t and document d from a catalog of N descriptions:

$$\text{tfidf}(t, d) = tf(t, d) \times \log\left(\frac{N}{df(t)}\right) \quad (4.1)$$

where $tf(t, d)$ is the frequency of t in d , and $df(t)$ is the number of descriptions containing t . Equation (4.1) down-weights common words (e.g. “smartphone”) and up-weights distinctive terms, giving the text vector $\mathbf{t}_i$.

Categorical block (one-hot encoding). Category, sub-category, brand, and OS become a binary indicator vector $\mathbf{c}_i$.

Specification block (min-max scaling). Eight numerical fields (price, RAM, storage, display size, refresh rate, battery capacity, weight, rating) are rescaled to $[0, 1]$ so no single field dominates by scale:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (4.2)$$

giving the numeric vector $\mathbf{s}_i$.

Weighted concatenation.

$$\mathbf{v}_i = \left[w_{\text{text}} \cdot \mathbf{t}_i \parallel w_{\text{cat}} \cdot \mathbf{c}_i \parallel w_{\text{spec}} \cdot \mathbf{s}_i \right] \quad (4.3)$$

where $\parallel$ is concatenation and w_{text} , w_{cat} , w_{spec} are the block weights from Section 4.1.4.

Given vectors $\mathbf{A}$ and $\mathbf{B}$:

$$\text{similarity}(\mathbf{A}, \mathbf{B}) = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \cdot \|\mathbf{B}\|} \quad (4.4)$$

with a score in $[0, 1]$: 1 means identical attribute space, 0 means no shared attributes.

Algorithm 1 Content-Based Product Recommendation using Cosine Similarity**Require:** Product catalog $\mathcal{P} = \{p_1, p_2, \dots, p_m\}$, viewed product p_q **Ensure:** Ordered list of top- N recommended products**Step 1: Feature Vector Construction**for each product $p_i \in \mathcal{P}$ do

Encode category, sub-category, brand, and OS as one-hot vectors

 Normalize price and specification fields to $[0, 1]$ using min-max scaling

Compute TF-IDF vector from product description

 Concatenate weighted features: $\mathbf{v}_i \leftarrow [w_{\text{text}} \cdot \text{TF-IDF} \parallel w_{\text{cat}} \cdot \text{category} \parallel w_{\text{spec}} \cdot \text{specs}]$

end for

Step 2: Similarity ComputationRetrieve feature vector $\mathbf{v}_q$ for the viewed product p_q for each product $p_i \in \mathcal{P}, p_i \neq p_q$ do

$$s_i \leftarrow \frac{\mathbf{v}_q \cdot \mathbf{v}_i}{\|\mathbf{v}_q\| \cdot \|\mathbf{v}_i\|}$$

end for

Step 3: Ranking and SelectionSort all products by s_i in descending orderReturn top- N products (default $N = 5$) as recommendations**Weight Selection and Validation**

The weights are chosen by a grid search scored against a relevance rule: two products are relevant if they share a category and satisfy at least two of three conditions, comparable price, comparable specifications, and overlapping description keywords. This majority-vote rule replaces a simpler same-category-only rule, since the category vector is itself one of the weighted inputs; scoring purely against category membership would reward the search for leaning on that vector regardless of actual recommendation quality (“label circularity”).

For a candidate weight triple, vectors are built with Equation (4.3) over a sampled subset of the catalog, pairwise similarities computed with Equation (4.4), and the candidate scored by the Pearson correlation between similarity values and the binary relevance

labels y :

$$\text{score}(w_{\text{text}}, w_{\text{cat}}, w_{\text{spec}}) = \frac{\text{Cov}(\text{sim}, y)}{\sigma_{\text{sim}} \sigma_y} \quad (4.5)$$

The search selects the weight triple maximizing Equation (4.5), normalized so the weights sum to one:

$$w'_i = \frac{w_i}{w_{\text{text}} + w_{\text{cat}} + w_{\text{spec}}}, \quad i \in \{\text{text}, \text{cat}, \text{spec}\} \quad (4.6)$$

To confirm the result was not a fragile, one-off outcome, a sensitivity sweep varied each weight across a range of values while holding the ratio of the other two fixed, re-scoring recommendation quality with Equation (4.5) at each point. This sweep and its results are presented in Section 5.2.

Evaluation Metrics

The tuned engine is benchmarked against a random-recommendation baseline using five standard ranking metrics, computed against the same majority-vote relevance labels used in Equation (4.5). For a query product q with a top- K list and relevant-item set R_q :

$$\text{Precision@}K = \frac{|\{\text{top-}K \text{ recommendations for } q\} \cap R_q|}{K} \quad (4.7)$$

$$\text{Recall@}K = \frac{|\{\text{top-}K \text{ recommendations for } q\} \cap R_q|}{|R_q|} \quad (4.8)$$

Because $|R_q|$ often exceeds K , no recommender can reach a perfect Recall@K; the theoretical ceiling is:

$$\text{Recall@}K_{\max} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{\min(K, |R_q|)}{|R_q|} \quad (4.9)$$

$$\text{HitRate@}K = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \mathbb{1}_{\left[|\{\text{top-}K \text{ recommendations for } q\} \cap R_q| > 0\right]} \quad (4.10)$$

$$\text{MRR} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{1}{\text{rank}_q} \quad (4.11)$$

$$\text{NDCG@K} = \frac{\sum_{i=1}^K \frac{rel_i}{\log_2(i+1)}}{\sum_{i=1}^{|R_q|} \frac{1}{\log_2(i+1)}} \quad (4.12)$$

Precision@K measures the fraction of top- K recommendations that are relevant; Recall@K the fraction of relevant items captured; HitRate@K whether at least one hit occurred; MRR how close the first relevant item sits to rank 1; and NDCG@K rewards both including and correctly ranking relevant items, normalized to $[0, 1]$.

Implementation

The feature matrix is pre-computed over the full catalog at server startup and cached in memory using Scikit-learn's `TfidfVectorizer`, `OneHotEncoder`, `MinMaxScaler`, and `cosine_similarity`. On a recommendation request, the system looks up the pre-computed similarity row, sorts scores, filters out-of-stock items, and returns the top- N product IDs, which are used to query MySQL for full product details serialized back to the frontend.

4.2 Incremental Development Model

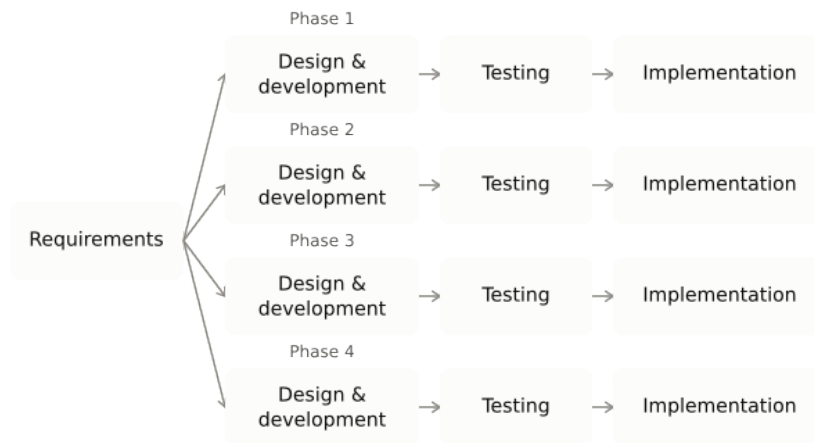


Figure 4.5: Incremental Development Model Diagram

Development followed an Incremental model, with a prioritized feature backlog delivered across four phases, each producing a working software increment:

- **Phase 1:** Setup, database schema, authentication (registration, login, JWT), role-based access control.
- **Phase 2:** Product management, category/brand management, seller dashboard, admin product oversight.
- **Phase 3:** Cart, order placement/tracking, payment gateway integration.
- **Phase 4:** Recommendation engine implementation and validation, semantic search with price filtering, CNN-based visual search (OpenCV image preprocessing via a pre-trained MobileNetV2 model, and cosine similarity matching against catalog image embeddings), and final testing and deployment.

4.3 Dataset Description

4.3.1 Dataset

The recommendation engine uses a dataset of approximately 10,000 entries (roughly 6,000 smartphones, 4,000 laptops), each with a product identifier, name, category, sub-category, brand, OS, free-text description, and numerical fields: price (NPR), RAM (GB), storage (GB), display size (inches), refresh rate (Hz), battery capacity (mAh), weight (grams), and rating.

The dataset was built by combining two separately sourced datasets – one containing smartphone listings and the other containing laptop listings – into a single unified dataset matching the exact column specification above. Both source datasets were mapped onto the same schema so that every row, regardless of whether it originally came from the smartphone or laptop collection, carries the same set of fields.

Table 4.1: Description of the combined dataset columns.

Column	Meaning
Product ID	Unique identifier of each product in the dataset.
Name	Name of the product (smartphone or laptop).
Category	Top-level category, e.g., Smartphone or Laptop.
Sub-category	Finer classification within the category (e.g., Gaming Laptop, Flagship Phone).
Brand	Manufacturer or brand name of the product.
OS	Operating system of the product (e.g., Android, iOS, Windows).
Description	Free-text product description used for TF-IDF vectorization.
Price (NPR)	Price of the product in Nepalese Rupees.
RAM (GB)	Random access memory capacity.
Storage (GB)	Internal storage capacity.
Display Size (inches)	Screen size of the product.
Refresh Rate (Hz)	Display refresh rate.
Battery Capacity (mAh)	Battery capacity of the product.
Weight (grams)	Physical weight of the product.
Rating	User/aggregate rating score of the product.

A sample of the combined dataset is provided in Appendix B.

Rows with missing values were dropped before feature extraction. Descriptions were vectorized with TF-IDF; category, sub-category, brand, and OS with one-hot encoding; numeric fields with min-max scaling. Because a full pairwise search across all weight combinations is expensive at 10,000 rows, the weight search and its evaluation (Chapter 5) were run on a stratified sample of a few hundred products, with the resulting weights applied to the full catalog.

4.3.2 Visual Search Dataset

For visual search, product images were sourced from the **Amazon Electronics Products 10k Items – 2023** dataset. Of the approximately 10,000 product records in the full catalog, only 428 had valid, accessible image URLs; the remaining entries were excluded

due to missing or broken links. The visual search module therefore operates on this filtered subset of 428 products, with each image mapped one-to-one to its corresponding catalog entry via product ID. A sample of this dataset is provided in Appendix B.

Each image is resized to 224×224 pixels and pixel values are normalized to match the input requirements of the feature extraction model. The image is then passed through a pretrained MobileNetV2 network, with its classification layer removed, to obtain a fixed-length feature embedding capturing visual characteristics such as shape, color, and texture. Embeddings for all 428 catalog images were precomputed and stored offline, so that at query time only the uploaded image needs to be processed and compared against the stored embeddings using cosine similarity to retrieve the most visually similar products.

4.4 Tools Used

Category	Technology	Purpose
Frontend	React.js, Tailwind CSS	User interface development
Backend	Django, Django REST Framework	API development and business logic
Database	MySQL	Relational data storage and management
AI / ML	Scikit-learn, TensorFlow/Keras (MobileNetV2), Sentence-Transformers, OpenCV	Recommendation engine, visual image search, and semantic search
Authentication	JSON Web Tokens (JWT)	User authentication and secure session management
Payment	Khalti Sandbox	Payment integration and test transaction processing
Version Control	Git, GitHub	Source code management and version control
Project Management	Feature Backlog (Phase-Based Tracking)	Task prioritization and progress tracking across development phases
IDE	Visual Studio Code	Software development and code editing
API Testing	Postman	API endpoint testing and documentation

Table 4.2: Development Tools and Technologies

CHAPTER 5:

RESULTS AND ANALYSIS

This chapter presents the work completed on GadgetHub across the backend, the three intelligent product discovery features, the frontend, and the version-control workflow, followed by the verification and validation carried out on the completed system.

5.1 Backend Development (Django REST Framework)

- Set up the Django project with MySQL integration, using an `.env` file to keep credentials and secrets out of source control.
- Designed the schema across the catalog and accounts apps, matching the ER diagram: User, Seller, Product, Brand, Category.
- Built a custom User model with a role field (Customer, Seller, Admin), set up before the first migration to avoid switching User models later.
- Implemented JWT authentication via `djangorestframework-simplejwt`: registration, login, and token validation on protected endpoints.
- Implemented role-based permission classes (`IsSeller`, `IsAdmin`) restricting access by role.
- Built the seller onboarding workflow: a seller registers, enters a pending state, and is admin-approved before listing products, enforced at the API level.
- Built seller-side product management: add a new product, or claim an unclaimed catalog listing under a custom price and stock quantity.
- Developed catalog APIs: category/brand listing, keyword search with price-range filtering, and a featured-products endpoint balancing phones and laptops.
- Set up media and static file handling for product images.
- Resolved a MySQL connection failure caused by special characters in the password (URL-encoding) and a migration conflict from a duplicate model definition.
- Implemented the shopping cart API: add/update/remove items, quantity checks against live stock, and cart persistence per user.

- Implemented order placement and tracking: order creation from cart contents, order status updates, and order history endpoints for both customers and sellers.
- Integrated the Khalti payment gateway for transaction processing, including payment initiation, verification callback handling, and marking orders as paid/unpaid based on gateway response.
- Integrated email notifications for order confirmation and status updates.

5.2 Recommendation Engine Development and Evaluation

- Built the core pipeline: TF-IDF, one-hot encoding, and min-max scaling combined via weighted concatenation and scored with cosine similarity, precomputed once at startup.
- Identified and fixed a circularity issue in the initial weight search (a same-category-only relevance proxy), replacing it with the majority-vote relevance rule described in Section 4.1.4.
- Under the corrected rule, the grid search selected $w_{\text{text}} = 0.692$, $w_{\text{cat}} = 0.231$, $w_{\text{spec}} = 0.077$, text-dominant, since numeric specs alone already separate laptops from phones almost as cleanly as category, adding little once category is accounted for.
- An independent sensitivity sweep (each weight varied 0.1–0.9, others held at baseline ratio) confirmed the result was not a fluke: text peaked near its selected value, category stayed flat, and spec declined monotonically, all consistent with the grid search’s choice.

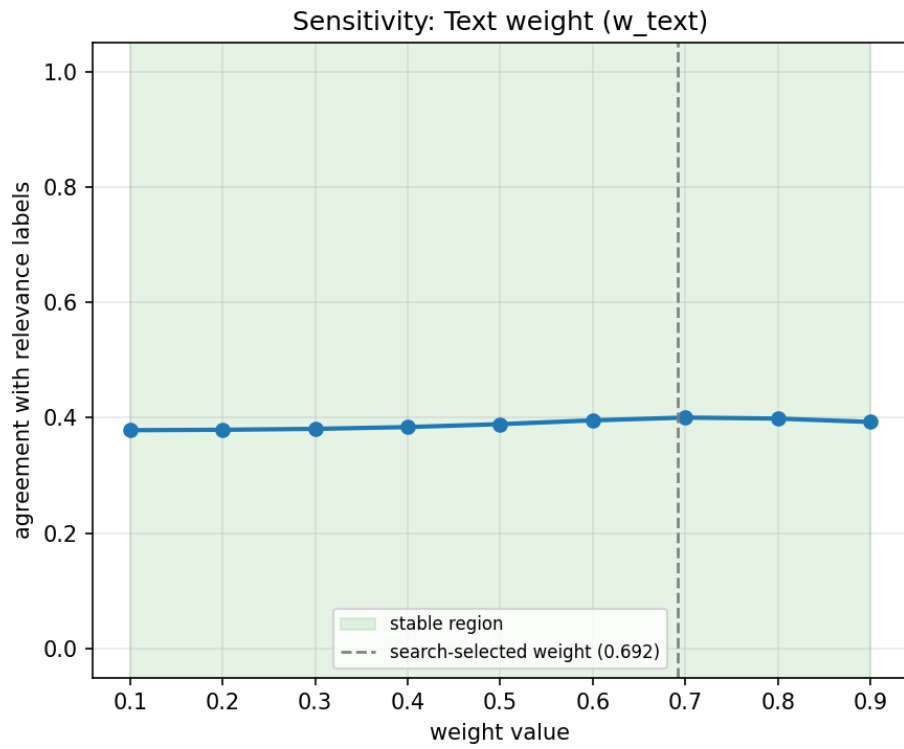


Figure 5.1: Weight Sensitivity Analysis – Text Weight

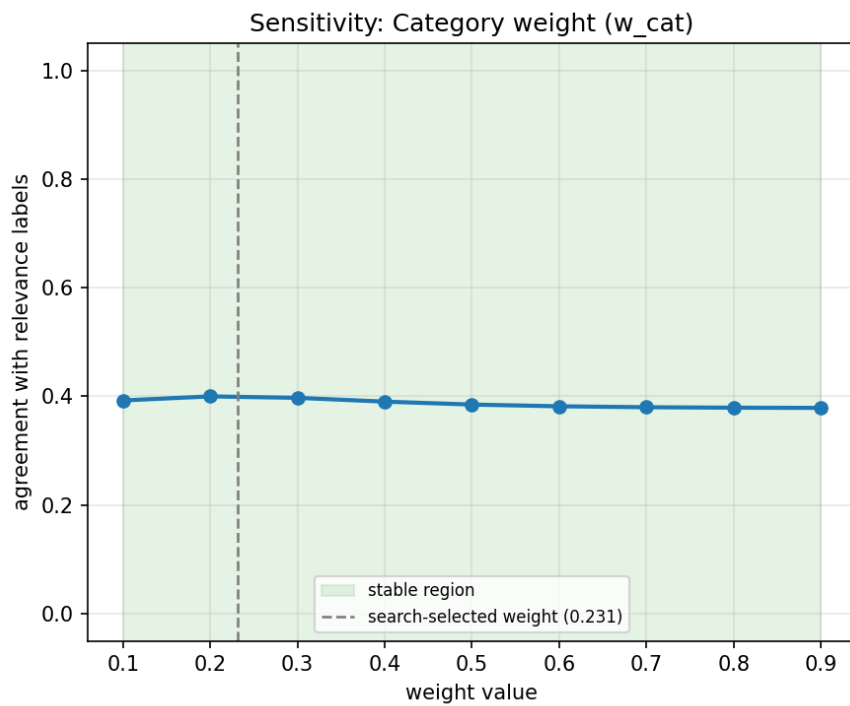


Figure 5.2: Weight Sensitivity Analysis – Category Weight

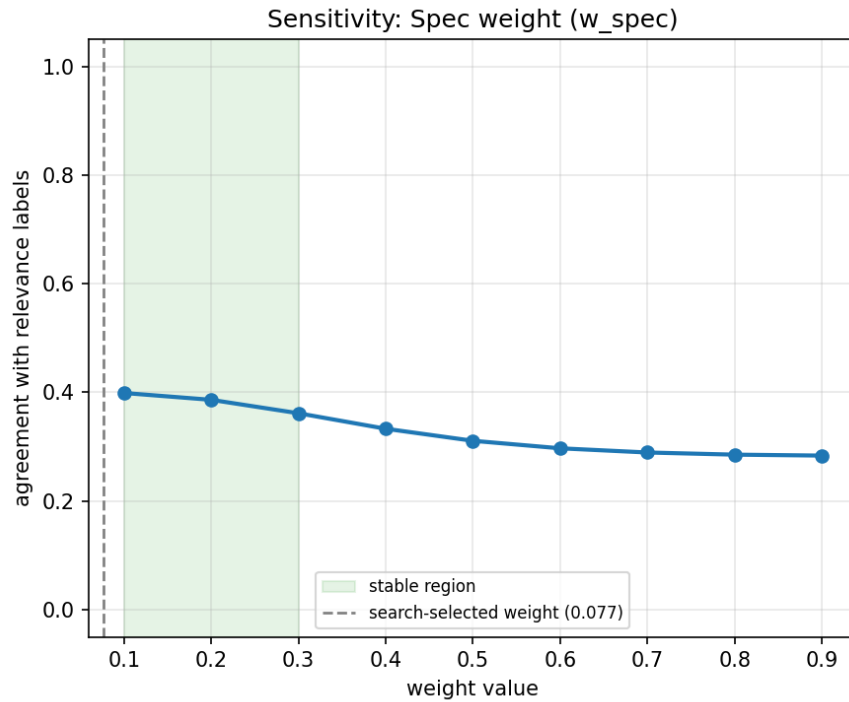


Figure 5.3: Weight Sensitivity Analysis – Spec Weight

- Benchmarked against a random-recommendation baseline on the same relevance rule: Precision@5 0.645 vs 0.057, Hit Rate@5 0.950 vs 0.251, MRR 0.864 vs 0.170, NDCG@5 0.682 vs 0.057.

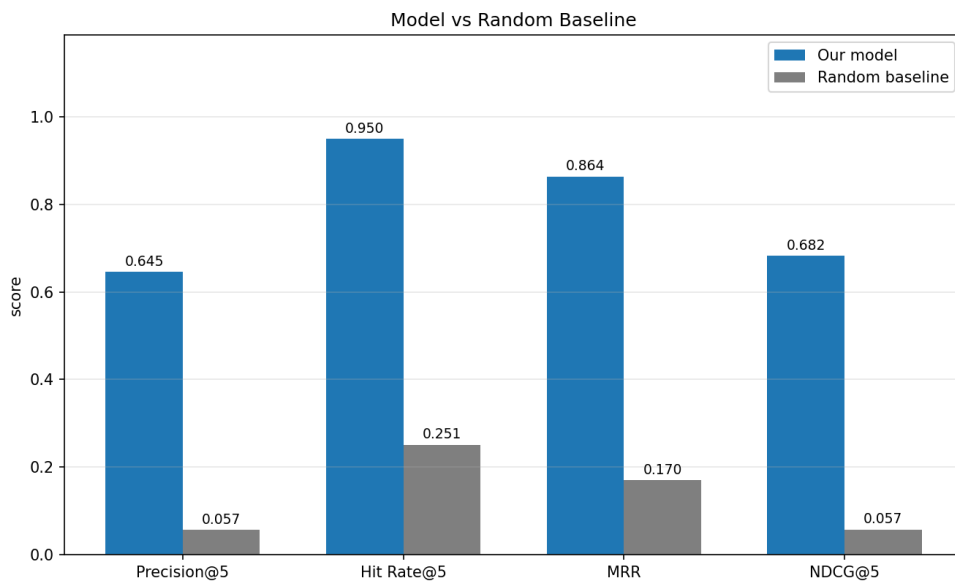


Figure 5.4: Recommendation Quality: Model vs Random Baseline

- Raw Recall@5 was initially low (0.125). This traced to the relevant-item pool

being large relative to $k = 5$: Equation (4.9) gives a theoretical ceiling of ~ 0.247 for this dataset, so the engine reaches $\sim 51\%$ of the maximum achievable.

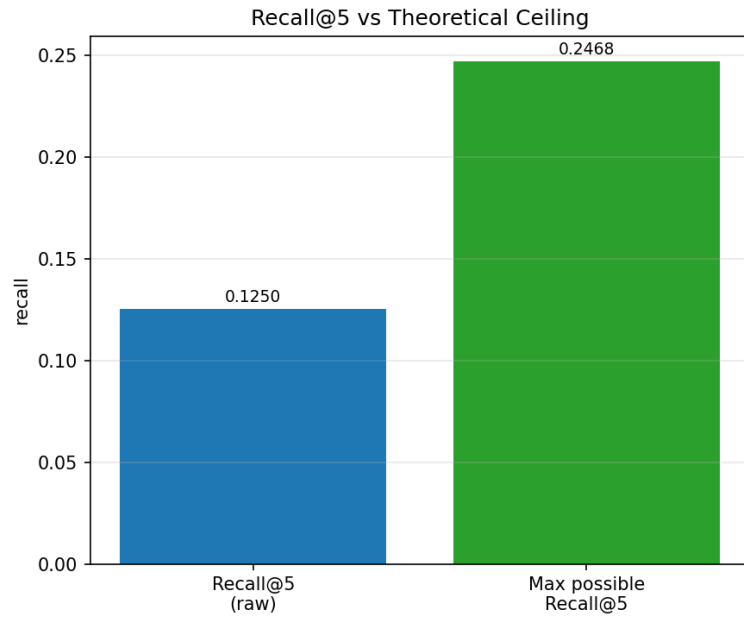


Figure 5.5: Recall@5 versus its Theoretical Ceiling

- Ran an end-to-end demonstration (5 laptops + 5 smartphones, top-5 each) on the full 10,000-row catalog, confirming the pipeline works outside the smaller tuning sample (terminal output in Appendix C).

5.3 Visual Search Development and Evaluation

- Built an image preprocessing pipeline using OpenCV (face detection and masking for privacy, resizing, and color-space conversion), followed by feature extraction using a pretrained MobileNetV2 CNN (classification layer removed, average pooling), producing a 1280-dimensional embedding per image.
- Embeddings were precomputed for the primary 10,000-product catalog (smartphones and laptops) and, to extend coverage to related accessories (e.g., bags, AirPods, watches), for a supplementary 429-product Amazon Electronics dataset. Visual search queries match against both combined.
- At query time, the uploaded image is embedded using the same pipeline and compared against cached embeddings via cosine similarity.

- Quantitative evaluation was performed on the 10,000-product catalog, the only dataset with the structured attributes (category, price, specs) needed to construct an automated relevance ground truth. The 429-product accessory set extends functional coverage but was not separately benchmarked.
- Benchmarked on a 400-product sample against two baselines using the same four ranking metrics applied to the recommendation engine: Precision@5 0.328, Hit Rate@5 0.723, MRR 0.577, NDCG@5 0.357 for the model; vs. 0.066/0.285/0.192/0.069 for a pure-random baseline and 0.121/0.448/0.291/0.125 for a category-restricted random baseline. The model outperforms both baselines on every metric.

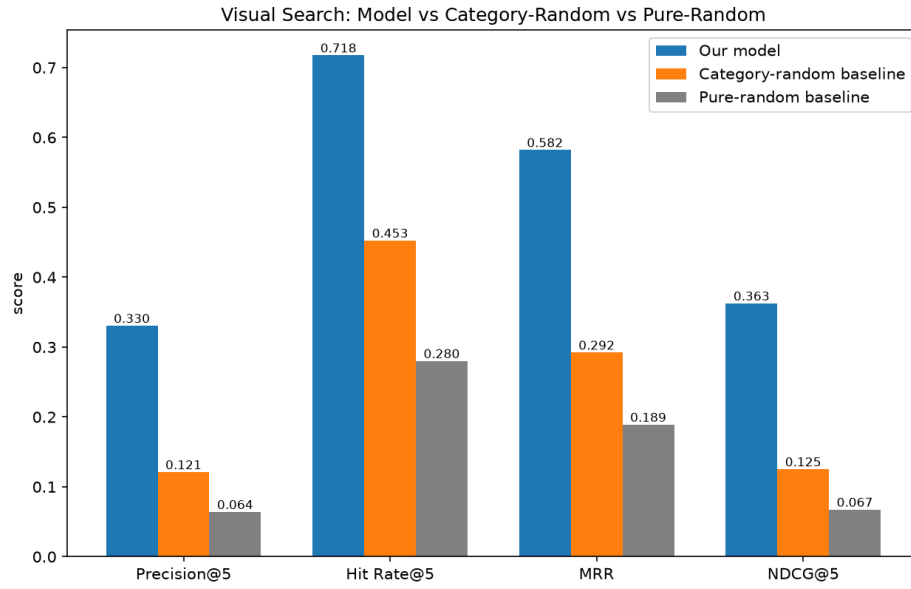


Figure 5.6: Visual Search Quality: Model vs Category-Random vs Pure-Random Baseline

- Raw Recall@5 (0.0662) is low because the relevant-item pool exceeds $k = 5$ for most queries; against the theoretical ceiling of 0.3168, the model reaches roughly 21% of the maximum achievable recall.

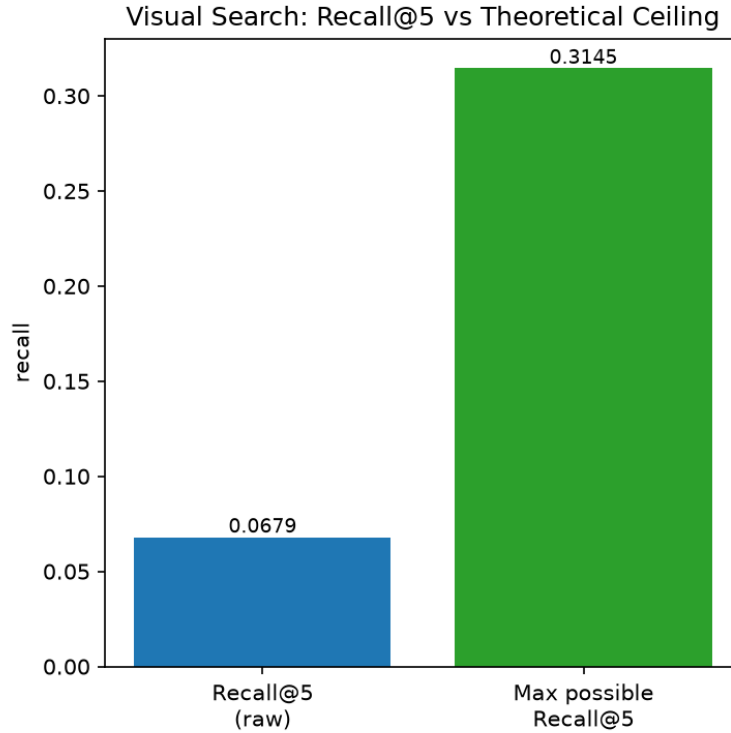


Figure 5.7: Visual Search Recall@5 versus its Theoretical Ceiling

5.4 Semantic Search Development and Evaluation

- Implemented query preprocessing (cleaning, tokenization, normalization) and integrated a pretrained Sentence-Transformer model (all-MiniLM-L6-v2) to generate 384-dimensional embeddings for query and product text (name, brand, description).
- Embeddings were precomputed for both the 10,000-product catalog and the 429-product accessory dataset, so queries for phones/laptops and for broader electronics/accessories are both matched. Results are ranked by cosine similarity and further refined with price-range and category filtering.
- As with visual search, quantitative evaluation used the 10,000-product catalog for the same structured-attribute reason.
- Benchmarked on the same 400-product sample: Precision@5 0.412, Hit Rate@5 0.818, MRR 0.652, NDCG@5 0.438 for the model; vs. 0.066/0.285/0.192/0.069 (pure-random) and 0.121/0.448/0.291/0.125 (category-random). Semantic search outperforms visual search on every metric here, expected since the relevance

ground truth partly reflects keyword overlap (favoring text-based retrieval), and Sentence-BERT is purpose-built for semantic text similarity while MobileNetV2 is a general-purpose image classifier repurposed as a feature extractor.

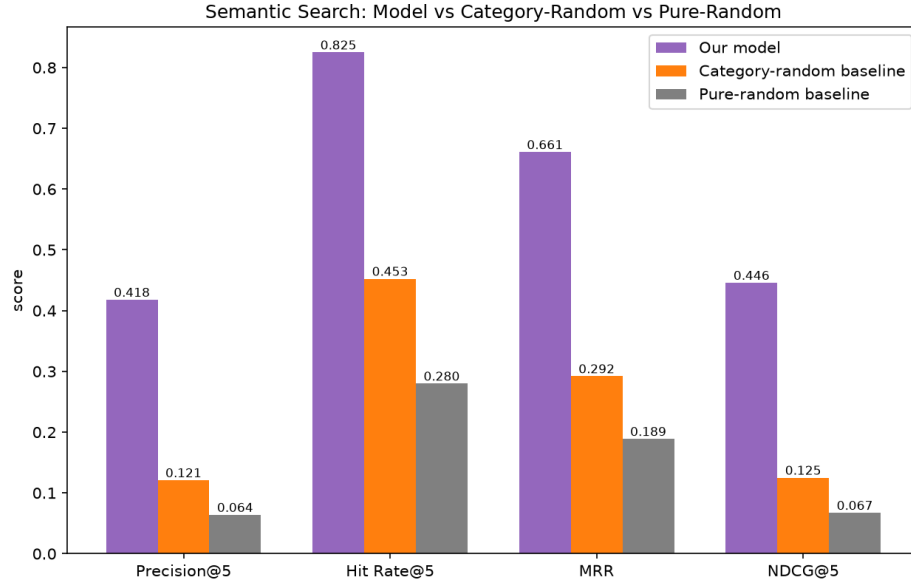


Figure 5.8: Semantic Search Quality: Model vs Category-Random vs Pure-Random Baseline

- Raw Recall@5 (0.0868) is likewise limited by the $k = 5$ cutoff; against the theoretical ceiling of 0.3168, the model reaches roughly 27% of the maximum achievable recall.

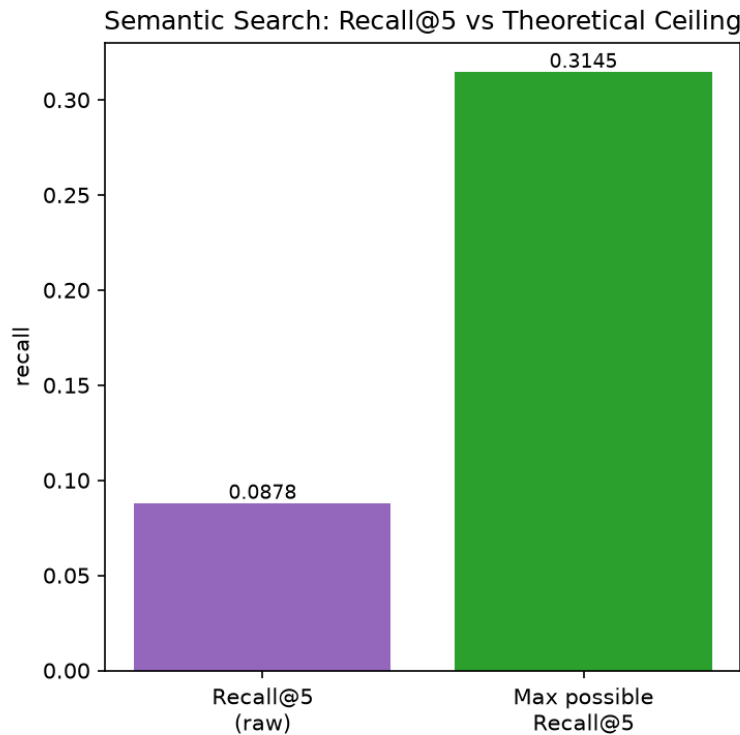


Figure 5.9: Semantic Search Recall@5 versus its Theoretical Ceiling

5.5 Frontend Development (React with Tailwind CSS)

- Built a combined login/registration page with a role selector (Customer or Seller), connected to the JWT endpoints.
- Built the customer onboarding preference page (category interest, budget range, priority specification), saved to the account.
- Built the customer homepage, fetching personalized recommendations from saved preferences.
- Built the product detail page: specifications, description, pricing, stock, and a similar-products section powered by the recommendation engine.
- Built the cart page: item listing, quantity adjustment, stock validation, and order summary.
- Built the checkout page and integrated the Khalti payment portal, including handling payment success and failure states.
- Built the customer order history page, showing paid/unpaid and in-progress orders.

- Built the visual search interface: photo upload, preview, and a results grid displaying visually matched products.
- Built the semantic search bar with live query submission and a faceted-filter panel (price, category, brand).
- Built the seller onboarding page for business name and contact information.
- Built the seller dashboard: add-product form, search-and-claim interface, live view of listed products, and a verification status banner.

5.6 Version Control and Team Collaboration

- Set up Git with the team's shared GitHub repository.
- Excluded a large, regenerable similarity-matrix artifact from version control via `.gitignore` rather than forcing it through.
- Pushed the complete backend and frontend implementation to a shared feature branch for review.

5.7 Verification

- Every API endpoint was manually tested in Postman before frontend integration, checking request/response formats and status codes.
- Role-based access was verified with and without valid JWTs, and across roles, confirming unauthorized requests return 401/403 rather than succeeding.
- Database migrations were checked against the live MySQL schema after every model change, especially after the earlier migration conflict.
- The recommendation engine's relevance definition was verified for internal consistency, confirming the weight-search and evaluation scripts share one relevance function rather than each defining its own.
- The engine's output was checked to correctly exclude out-of-stock items and the viewed product itself.

5.8 Validation

- The full customer journey, registration through product detail viewing, was validated end-to-end.
- The full seller journey, registration, pending state, admin approval, listing, was validated end-to-end.
- The personalized homepage was validated across category, budget, and priority-spec combinations, catching and fixing an edge case where selecting “Both” categories returned only one product type.
- The tuned weights were validated independently of the grid search itself, through the sensitivity analysis and random-baseline comparison in Section 5.2.
- Product images were validated across the seller dashboard, homepage, and detail page after a media path mismatch was found and corrected.
- Multiple test accounts across both roles were used to repeatedly walk through every implemented flow before code was pushed.

CHAPTER 6:

FUTURE ENHANCEMENTS

While GadgetHub currently meets its planned objectives, several enhancements have been identified for future work:

- A hybrid recommendation approach will be explored, combining the current content-based method with collaborative filtering once sufficient user interaction data has accumulated.
- The visual search image dataset, currently limited to 428 products, will be expanded to cover a larger portion of the catalog.
- The initial smartphone and laptop dataset will be gradually supplemented with real listings from verified sellers as the platform is deployed.
- A seller-side analytics dashboard will be added, showing revenue trends, order volume, and low-stock alerts.
- A customer review and rating system will be introduced to improve trust and to support collaborative filtering in future iterations.
- The recommendation, search, and visual search pipelines will be extended to additional electronics categories such as tablets, smartwatches, and accessories.
- A dedicated mobile application will be developed to complement the existing web platform.
- Additional payment gateways will be integrated alongside Khalti to widen payment options for customers.

CHAPTER 7:

CONCLUSION

This report has presented the design, implementation, and evaluation of GadgetHub, an AI-enabled multi-vendor electronics marketplace. The platform combines core e-commerce functionality, JWT-based authentication and role-based access control, seller onboarding with administrative approval, cart and order management, and Khalti payment integration, with three intelligent product discovery features: a content-based recommendation engine using weighted cosine similarity over TF-IDF, one-hot, and min-max scaled features; a semantic search interface using Sentence-Transformer embeddings; and a CNN-based visual search pipeline using MobileNetV2 feature embeddings.

All three intelligent features were implemented, benchmarked against random baselines, and validated, alongside the complete customer, seller, and administrator workflows. The recommendation engine's weight-selection methodology was corrected after an internal circularity issue was identified, and the resulting weights were independently verified through a sensitivity analysis. These results demonstrate that the proposed architecture is sound, that the content-based approach addresses the cold-start problem inherent to a newly launched marketplace, and that GadgetHub provides a practical, cost-effective platform for intelligent, multi-vendor electronics retail in the Nepali market.

REFERENCES

- [1] S. H. Choi, H. J. Kim, and J. H. Lee, “Semantic product search over structured e-commerce catalogs,” *Information Processing & Management*, vol. 58, no. 4, 2021.
- [2] J. Kim, Y. Lee, and S. Park, “A product information meta-search framework for electronic commerce,” in *Proc. International Conference on Electronic Commerce*, 2000.
- [3] P. Lops, M. de Gemmis, and G. Semeraro, “Content-based recommender systems: State of the art and trends,” in *Recommender Systems Handbook*, F. Ricci, L. Rokach, and B. Shapira, Eds. Boston, MA, USA: Springer, 2015, pp. 73–105.
- [4] G. Salton, A. Wong, and C.-S. Yang, “A vector space model for automatic indexing,” *Communications of the ACM*, vol. 18, no. 11, pp. 613–620, Nov. 1975.
- [5] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [6] J. Nigam, S. Bhatia, and A. Kumar, “Semantic matching for product search in e-commerce,” in *Proc. International Conference on Information Retrieval*, 2020.
- [7] Z. Liu, P. Luo, S. Qiu, X. Wang, and X. Tang, “DeepFashion: Powering robust clothes recognition and retrieval with rich annotations,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1096–1104.
- [8] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

APPENDICES

7.1 APPENDIX A: Technical Specifications

7.1.1 Software Requirements

Category	Technology
Frontend	React.js, Tailwind CSS
Backend	Django, Django REST Framework
Database	MySQL
AI / ML	Scikit-learn, TensorFlow/Keras (MobileNetV2), Sentence-Transformers, OpenCV
Authentication	JSON Web Token (JWT)
Payment Gateway	Khalti
Version Control	Git, GitHub
IDE & Testing	Visual Studio Code, Postman
Project Management	Feature backlog (phase-based tracking)

Table 7.1: Software Requirements of the Proposed System

7.1.2 Hardware Requirements

- Personal computer or laptop
- Intel Core i5 / AMD Ryzen 5 processor or higher
- Minimum 8 GB RAM
- Minimum 256 GB SSD storage
- Stable internet connection (minimum 10 Mbps recommended)

7.2 APPENDIX B: Dataset Sample

1	product_id	product_na	category	sub_categoria	brand	model	price_npr	processor	gpu	ram_gb	storage_gb	os	display_size	display_tpyr	display_resc	refresh_rate	rear_camer	front_camer
2	PHN-00001	Vivo V29	4G Smartphone	Budget	Vivo	Vivo V29	12500	Helio G85	Integrated	4	32	Android	6.4	PLS LCD	720x1560	90	48	13
3	PHN-00002	OnePlus No 15	Smartphone	Mid-Range	OnePlus	OnePlus No 15 P	65000	Helio G99	Integrated	8	64	Android	6.6	Super AMOLED	1440x3120	120	48	13
4	PHN-00003	iPhone 15	P Smartphone	Mid-Range	Apple	iPhone 15 P	74500	Apple A15 B	Integrated	6	128	iOS	6.5	OLED	1080x2400	120	48	32
5	PHN-00004	iPhone 15	6 Smartphone	Mid-Range	Apple	iPhone 15	35500	Apple A15 B	Integrated	6	128	iOS	6.5	Super Retin	1284x2778	90	50	32
6	LPT-00005	ASUS ROG Z	Laptop	Premium	ASUS	ROG Zephyr	445000	Apple M3 P	NVIDIA RTX	64	512	Windows 11	13.3	IPS	2560x1440	120	0	720
7	PHN-00006	Galaxy Z Fol	Smartphone	Mid-Range	Samsung	Galaxy Z Fol	42000	Snapdragon	Integrated	6	64	Android	6.7	IPS LCD	1284x2778	90	48	16
8	PHN-00007	iPhone SE 2i	Smartphone	Mid-Range	Apple	iPhone SE 2i	77000	Apple A15 B	Integrated	8	128	iOS	6.7	Super Retin	1080x2340	90	50	13
9	PHN-00008	Vivo X100 P	Smartphone	Budget	Vivo	Vivo X100 P	20000	Helio G37	Integrated	6	64	Android	6.1	PLS LCD	1080x2400	90	50	8
10	PHN-00009	Pixel 8 Pro	Smartphone	Mid-Range	Google	Pixel 8 Pro	57500	Dimensity 7	Integrated	6	64	Android	6.5	IPS LCD	1170x2532	90	48	8
11	LPT-00010	Acer Aspire	Laptop	Premium	Acer	Aspire 7	232000	Intel Core i5	Apple M3 P	32	512	Windows 11	14	IPS	3840x2400	120	0	1080
12	PHN-00011	Pixel 7a	8GE Smartphone	Flagship	Google	Pixel 7a	188500	Apple A16 B	Integrated	8	512	Android	6.8	LTPD OLED	1440x3120	144	50	32
13	PHN-00012	Nokia C22	3 Smartphone	Budget	Nokia	Nokia C22	20500	Helio G35	Integrated	3	32	Android	6.6	IPS LCD	720x1600	90	48	13
14	PHN-00013	OnePlus No 15	Smartphone	Mid-Range	OnePlus	OnePlus No 15	55500	Snapdragon	Integrated	12	256	Android	6.4	IPS LCD	1080x2400	90	108	16
15	LPT-00014	MSI Stealth	Laptop	Budget	MSI	Stealth 16	51000	Intel Core i5	AMD Radeon	8	128	Windows 11	15.6	VA	1366x768	60	0	1080
16	LPT-00015	Lenovo Legi	Laptop	Budget	Lenovo	Legion 5	57000	AMD Ryzen	AMD Radeon	4	256	Windows 11	15.6	VA	1920x1080	165	0	720
17	PHN-00016	OnePlus 12	Smartphone	Flagship	OnePlus	OnePlus 12	128500	Apple A17 P	Integrated	8	512	Android	6.9	AMOLED	1440x3120	120	64	32
18	LPT-00017	Dell Latitude	Laptop	Budget	Dell	Latitude 74	60000	AMD Athlon	AMD Radeon	4	128	Windows 11	14	IPS	1366x768	120	0	1080
19	LPT-00018	Acer Predat	Laptop	Premium	Acer	Predator H5	421000	Apple M3 M	Apple M3 M	16	2048	Windows 11	13.3	Retina XDR	2560x1440	144	0	720
20	PHN-00019	Galaxy M55	Smartphone	Budget	Samsung	Galaxy M55	16000	Helio G88	Integrated	3	64	Android	6.4	PLS LCD	720x1560	90	13	8
21	PHN-00020	Pixel 7a	6GE Smartphone	Mid-Range	Google	Pixel 7a	39000	Snapdragon	Integrated	6	64	Android	6.6	AMOLED	1284x2778	120	64	8
22	PHN-00021	Moto G54	Smartphone	Mid-Range	Motorola	Moto G54	76500	Snapdragon	Integrated	6	256	Android	6.7	Super AMOLED	1170x2532	120	64	8
23	PHN-00022	Vivo Y200	3 Smartphone	Budget	Vivo	Vivo Y200	24500	Helio G85	Integrated	3	32	Android	6.6	IPS LCD	720x1600	90	50	5
24	PHN-00023	Pixel 7a	16C Smartphone	Flagship	Google	Pixel 7a	172000	Snapdragon	Integrated	16	128	Android	7.6	Dynamic AM	1080x2340	144	200	12
25	PHN-00024	Oppe A18	8 Smartphone	Mid-Range	Oppe	Oppe A18	45500	Dimensity 6	Integrated	8	256	Android	6.4	Super AMOLED	1440x3120	120	64	16
26	LPT-00025	HP Spectre	Laptop	Premium	HP	Spectre x36	283000	AMD Ryzen	Apple M3 P	64	512	Windows 11	17.3	Mini-LED	3024x1964	60	0	720
27	LPT-00026	Lenovo Thin	Laptop	Budget	Lenovo	ThinkPad L1	51000	Intel Core i3	Integrated	8	128	Windows 11	15.6	TN	1366x768	90	0	1080
28	LPT-00027	Acer Nitro V	Laptop	Premium	Acer	Nitro V 15	480000	AMD Ryzen	NVIDIA RTX	64	1024	Windows 11	17.3	Mini-LED	2880x1800	120	0	720
29	LPT-00028	Lenovo Thin	Laptop	Mid-Range	Lenovo	ThinkPad X1	114000	Intel Core	UNNVIDIA RTX	8	256	Windows 11	16	IPS	2560x1600	90	0	1080

Figure 7.1: Sample of the 10,000-item combined product dataset used by the recommendation engine.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
· battery_ma	fast_chargir	weight	grar color	warranty	yr rating	num_rating	stock	quanti	seller_name	description	image	filen	image_path															
4947	18	181	Cream	2	4.2	2324	36	Bhaktapur	Vivo Vivo V	vivo-vivo-v2 product_images/vivo-vivo-v29.jpg																		
4564	45	177	Green	2	4.3	1967	16	NexGen Ele	OnePlus On	oneplus-onk product_images/oneplus-nord-n30.jpg																		
4664	18	172	Gold	1	4.4	2402	78	Subidha Dig	Apple iPhone	apple-iphon product_images/apple-iphone-15-plus.jpg																		
5368	25	205	Graphite	1	4.1	1329	54	Pokhara Ter	Apple iPhone	apple-iphon product_images/apple-iphone-15.jpg																		
74	100	2295	Platinum	1	4.9	690	22	Bhaktapur	ASUS ROG Z	asus-rog-zei product_images/asus-rog-zephyrus-g16.jpg																		
4675	67	199	White	2	3.5	1256	70	Tech Paradi	Samsung Ga	samsung-ga product_images/samsung-galaxy-z-fold5.jpg																		
4825	18	178	Titanium Blt	2	4	1416	105	Pokhara Ter	Apple iPhone	apple-iphon product_images/apple-iphone-se-2022.jpg																		
5219	10	183	Blue	1	4.1	228	10	Tech Paradi	Vivo Vivo X1	vivo-vivo-x1 product_images/vivo-vivo-x100-pro.jpg																		
5478	18	180	Red	1	3.5	1292	74	Bhaktapur	Google Pixe	google-pixe product_images/google-pixel-8-pro.jpg																		
80	140	2105	Lunar Gray	1	4.8	277	33	Biratnagar	Acer Aspire	acer-aspire- product_images/acer-aspire-7.jpg																		
4860	67	201	Cream	2	4.1	2477	65	GoDigital Ni	Google Pixe	google-pixe product_images/google-pixel-7a.jpg																		
5868	10	211	Silver	1	4	1883	149	SmartBuy N	Nokia Nokia	nokia-nokia product_images/nokia-nokia-c22.jpg																		
5408	45	183	White	1	3.9	2353	129	Digital Pasa	OnePlus On	oneplus-onk product_images/oneplus-nord-ce4.jpg																		
45	65	1672	Lunar Gray	1	3.3	1053	76	TechWorld	MSI Stealth	msi-stealth- product_images/msi-stealth-16.jpg																		
41	65	2046	Natural Silv	2	3.1	140	71	TechWorld	Lenovo Legi	lenovo-legic product_images/lenovo-legion-5.jpg																		
4815	80	189	Red	2	4.6	991	57	SmartBuy N	OnePlus On	oneplus-onk product_images/oneplus-nord-12.jpg																		
50	45	1588	Space Gray	2	4	818	34	Tech Paradi	Dell Latitudi	dell-latitude product_images/dell-latitude-7440.jpg																		
93	100	1779	Onyx Grey	1	4.7	755	18	GadgetGuru	Acer Predat	acer-predat product_images/acer-predator-helios-16.jpg																		
4998	18	192	Cream	2	3.2	1167	113	NexGen Ele	Samsung Ga	samsung-ga product_images/samsung-galaxy-m55.jpg																		
5045	45	172	Mint	1	4.1	370	13	ClickShop N	Google Pixe	google-pixe product_images/google-pixel-7a.jpg																		
5200	33	186	Gold	1	3.8	807	31	Tech Paradi	Motorola M	motorola-m product_images/motorola-moto-g54.jpg																		
5327	10	177	Silver	1	3.7	1128	85	Smart Gadg	Vivo Vivo Yz	vivo-vivo-yz product_images/vivo-vivo-y200.jpg																		
5060	80	221	Green	2	4.5	1060	11	GoDigital Ni	Google Pixe	google-pixe product_images/google-pixel-7a.jpg																		
4932	18	190	Cream	2	4.5	374	116	GoDigital Ni	Oppo Oppo	oppo-oppo- product_images/oppo-oppo-a18.jpg																		
76	100	1394	Onyx Grey	1	5	356	73	Nepal Elect	HP Spectre	hp-spectre- product_images/hp-spectre-x360-16.jpg																		
48	65	1614	Space Gray	2	4.1	633	52	MegaByte S	Lenovo Thir	lenovo-thin product_images/lenovo-thinkpad-114.jpg																		
88	130	2512	Onyx Grey	1	4.3	150	48	QuickTech	Acer Nitro	vacer-nitro-v product_images/acer-nitro-v-15.jpg																		
52	45	1443	Midnight Bli	1	4.3	516	15	TechWorld	Lenovo Thir	lenovo-thin product_images/lenovo-thinkpad-x1-carbon.jpg																		

Figure 7.2: Sample of the 10,000-item combined product dataset used by the recommendation engine.

	A	B	C	D	E
1	image_file	product_name	description	price_npr	
2	1318.jpg	Sony MDR-X	Sony MDR-X	3358	
3	9504.jpg	Cosmic Byte	Cosmic Byte	1118	
4	7222.jpg	realme Pad	realme Pad	25597	
5	5391.jpg	Fire-Boltt Ri	Fire-Boltt Ri	3678	
6	9166.jpg	XIRON 2 Pad	XIRON 2 Pad	2238	
7	8940.jpg	MOKOBARA	MOKOBARA	8798	
8	3708.jpg	DIGITEK® (L	DIGITEK® (L	1198	
9	3573.jpg	SEMINES Ta	SEMINES Ta	558	
10	683.jpg	CRUST CS3C	CRUST CS3C	1034	
11	9037.jpg	Redmi Pad	Redmi Pad	28798	
12	346.jpg	Lava Blaze 5	Lava Blaze 5	19198	
13	3038.jpg	Xiaomi Mi 4	Xiaomi Mi 4	2398	
14	9593.jpg	Amozo Ultra	Amozo Ultra	318	
15	4560.jpg	Portronics K	Portronics K	766	
16	8005.jpg	SeCro USB 3	SeCro USB 3	718	
17	9313.jpg	Carvaan Sar	Carvaan Sar	3984	
18	3985.jpg	Lapster usb	Lapster usb	318	
19	7955.jpg	Cosmic Byte	Cosmic Byte	2878	
20	1034.jpg	boAt Basshe	boAt Basshe	958	
21	6805.jpg	AGARO 3.1	AGARO 3.1	200	
22	8464.jpg	Boult Audio	Boult Audio	1758	
23	2150.jpg	HUMBLE M	HUMBLE M	158	
24	5404.jpg	Amazon Bra	Amazon Bra	222	
25	960.jpg	CLAVIER Ne	CLAVIER Ne	238	
26	2586.jpg	Razer Death	Razer Death	2158	
27	4934.jpg	POCO M4 P	POCO M4 P	2000	
28	1981.jpg	boAt Airdop	boAt Airdop	2398	
29	5491.jpg	CEDO Clear	CEDO Clear	318	

Figure 7.3: Sample of the Amazon Electronics Products 10k Items – 2023 dataset used for visual search.

7.3 APPENDIX C: Recommendation Engine Terminal Output

```
{
  "viewed_product": {
    "product_id": "PHN-06471",
    "product_name": "Pixel 8a 8GB/1024GB - Silver",
    "category": "Smartphone"
  },
  "recommendations": [
    {
      "id": "PHN-05544",
      "name": "Pixel 8 1024GB - Phantom Black",
      "category": "Smartphone",
      "similarity_score": 0.9943
    },
    {
      "id": "PHN-00700",
      "name": "Pixel 7a 1024GB - Cream",
      "category": "Smartphone",
      "similarity_score": 0.9928
    },
    {
      "id": "PHN-04723",
      "name": "Pixel 8 1024GB - Purple",
      "category": "Smartphone",
      "similarity_score": 0.9925
    },
    {
      "id": "PHN-09724",
      "name": "Pixel 7 Pro 1024GB - Red",
      "category": "Smartphone",
      "similarity_score": 0.9911
    },
    {
      "id": "PHN-01204",
      "name": "Pixel 7 Pro 1024GB - Black",
      "category": "Smartphone",
      "similarity_score": 0.9905
    }
  ]
}
```

Figure 7.4: Terminal Output: Top-5 Recommendation Demonstration

```
{
  "viewed_product": {
    "product_id": "PHN-01488",
    "product_name": "Realme Narzo 60 12GB/256GB - Green",
    "category": "Smartphone"
  },
  "recommendations": [
    {
      "id": "PHN-07890",
      "name": "Realme C67 256GB - Cream",
      "category": "Smartphone",
      "similarity_score": 0.9946
    },
    {
      "id": "PHN-08424",
      "name": "Realme C67 128GB - Purple",
      "category": "Smartphone",
      "similarity_score": 0.9935
    },
    {
      "id": "PHN-03010",
      "name": "Realme GT Neo5 512GB - Silver",
      "category": "Smartphone",
      "similarity_score": 0.991
    },
    {
      "id": "PHN-08785",
      "name": "Realme 12 Pro+ 128GB - Titanium Blue",
      "category": "Smartphone",
      "similarity_score": 0.9903
    },
    {
      "id": "PHN-03278",
      "name": "Realme C67 16GB/128GB - Silver",
      "category": "Smartphone",
      "similarity_score": 0.9893
    }
  ]
}
```

Figure 7.5: Terminal Output: Top-5 Recommendation Demonstration

7.4 APPENDIX D: User Interface Screenshots

7.4.1 Customer Interfaces

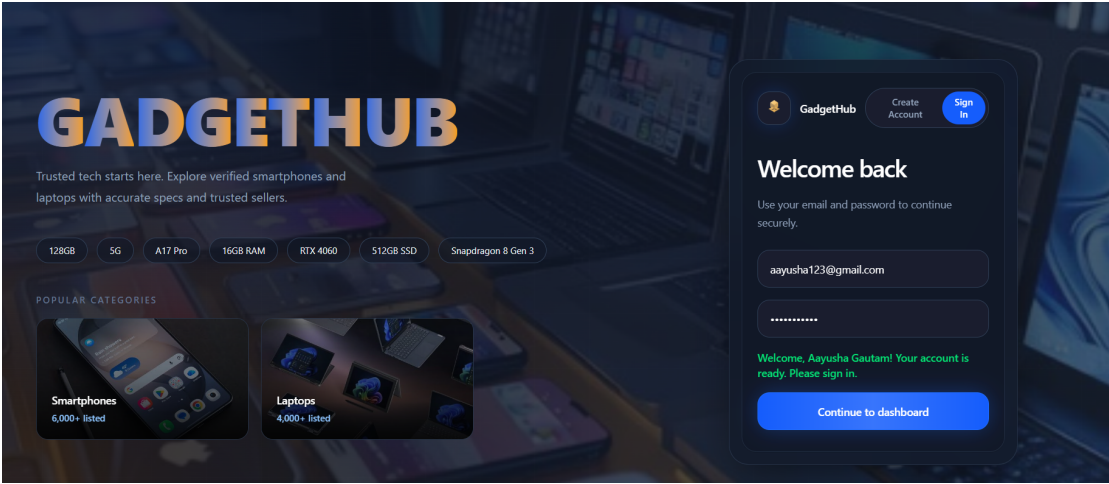


Figure 7.6: Sign In Interface

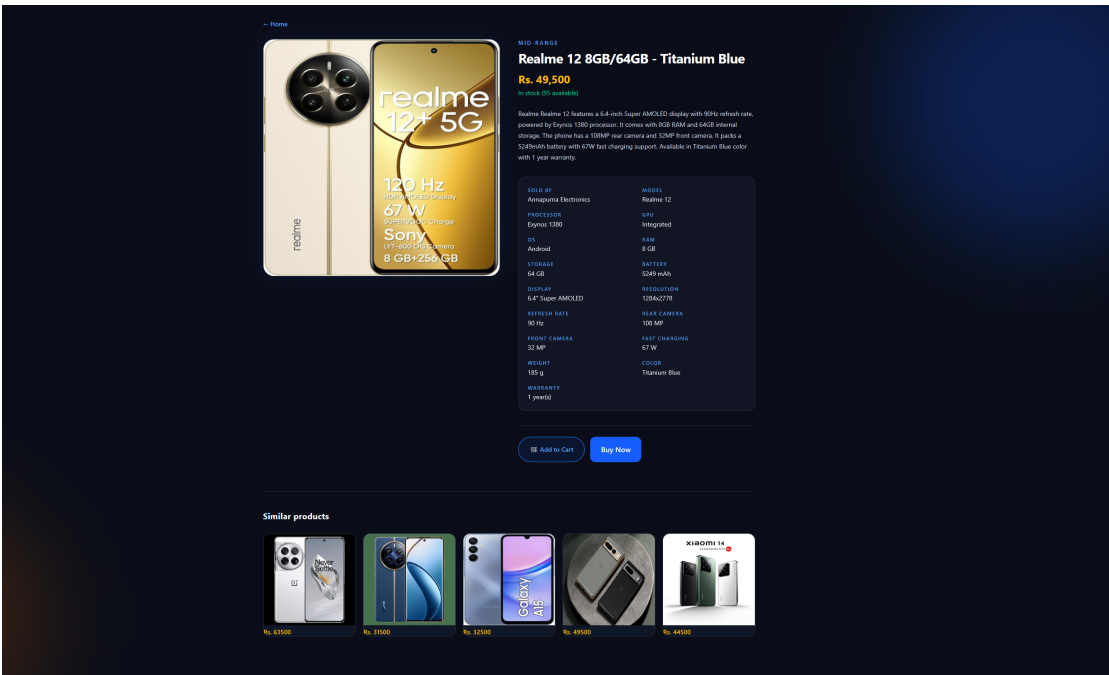


Figure 7.7: Product Detail Interface

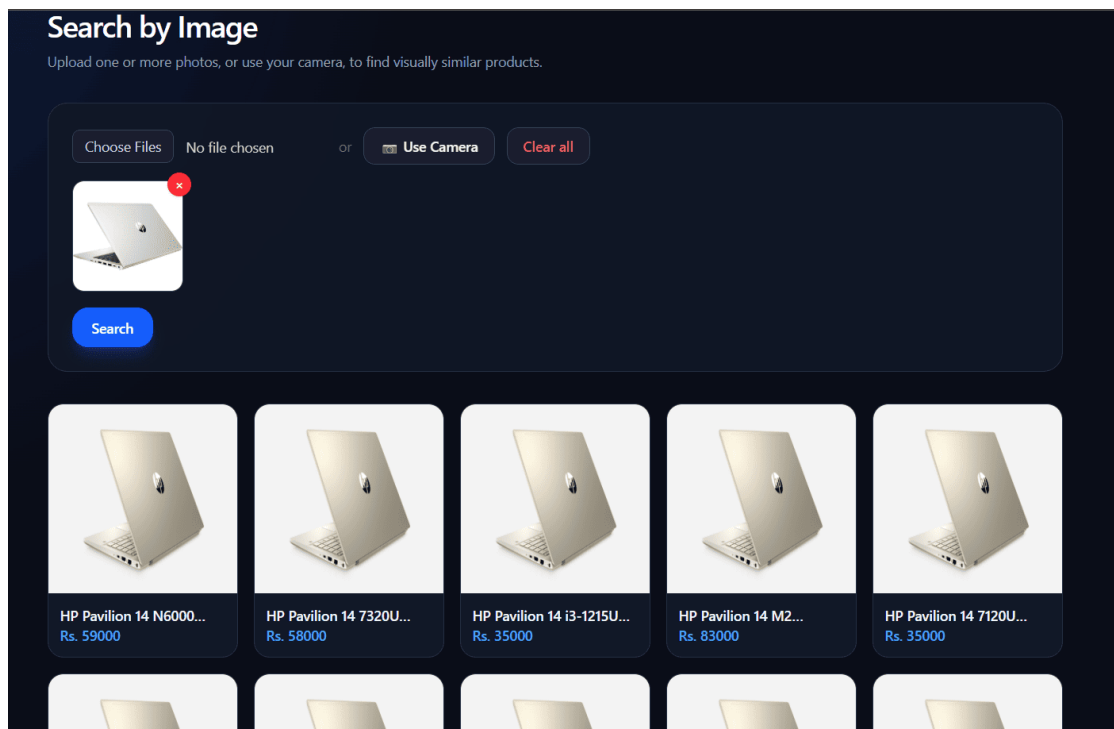


Figure 7.8: Visual Search Interface

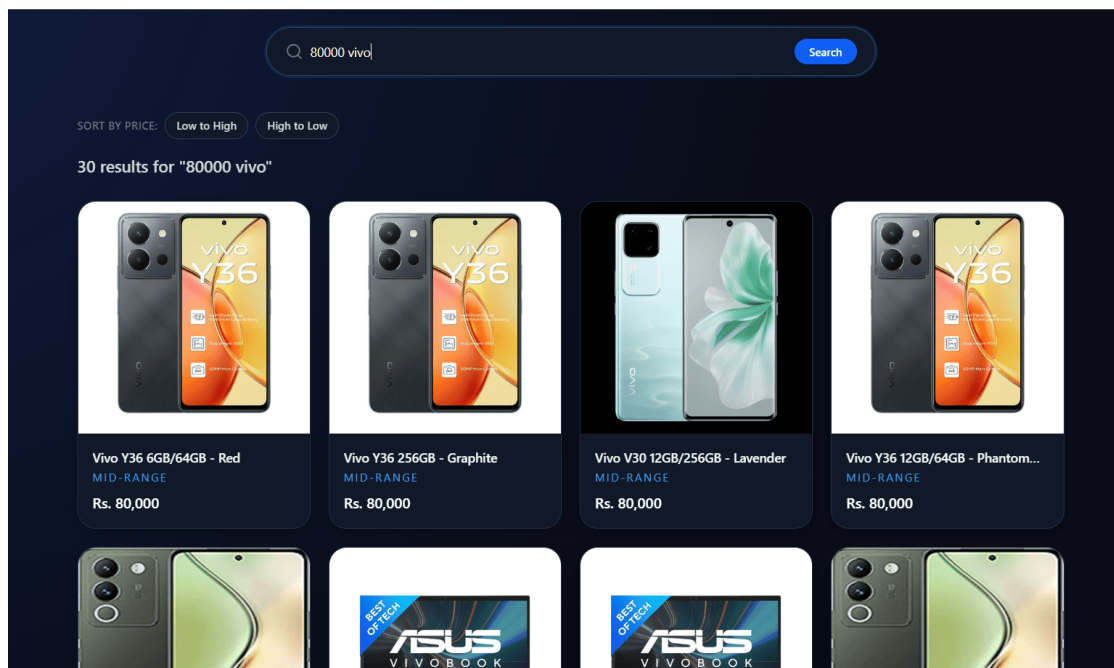


Figure 7.9: Semantic Search Interface

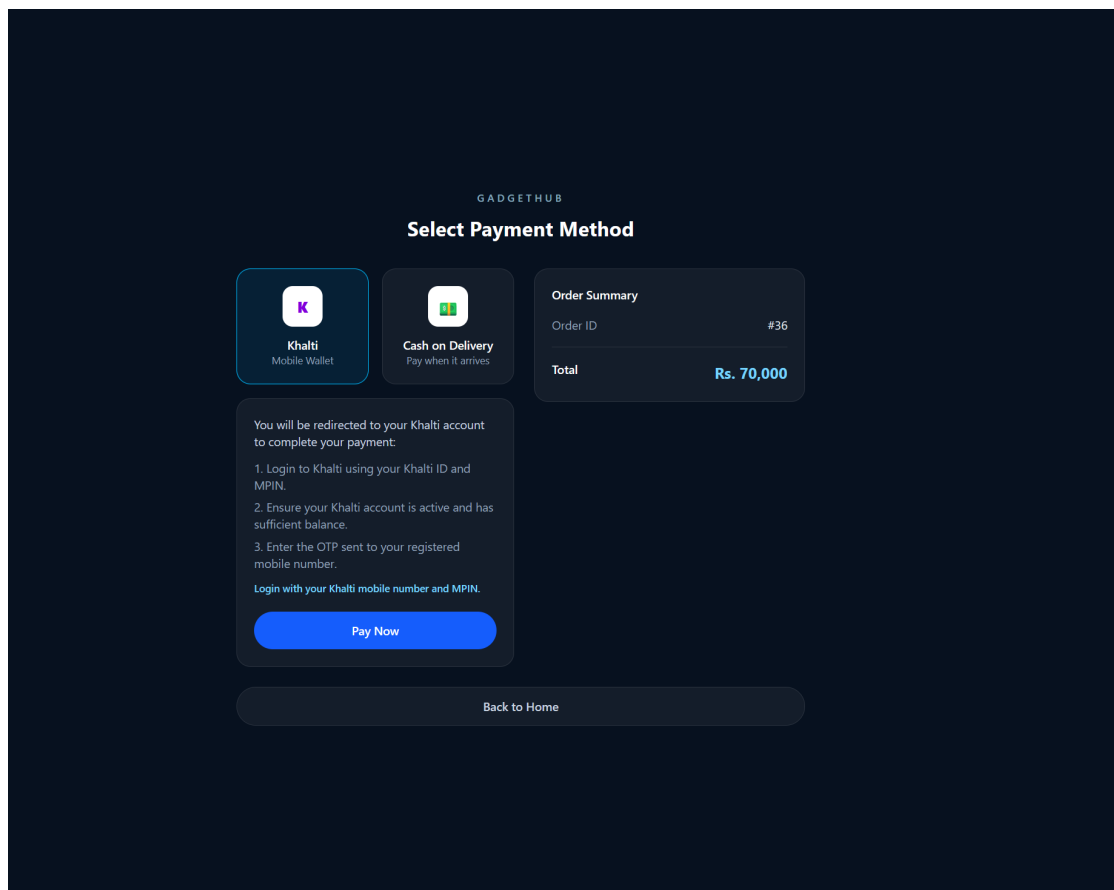


Figure 7.10: Checkout Page

Payment Details

This payment will expire on Aug 22, 2026 19:33 PM

Billed To:

Anshu

Himalaya College Of Engineering

Amount Summary:

Total Payable Amount

Rs 70000.00

PAYMENT POWERED BY

khalti

← Pay via Khalti Wallet

Enter Khalti ID

Country

Nepal

Khalti Mobile Number

9800000005

Khalti Password / MPIN

••••

Submit

Forgot your password? Reset Password

Cancel Payment

Figure 7.11: Khalti Payment Portal

7.4.2 Seller Interfaces

GadgetHub

Log out

Business Profile

BUSINESS NAME

ABC techstore

OWNER NAME

Aarush magar

CONTACT INFO

9877777777

EMAIL

aarush123@gmail.com

Edit profile

Your seller account is pending admin approval. You'll be able to add or claim products once approved.

My Products

Add Product

Browse & Claim

You haven't listed any products yet.

+ Add Product

Browse & Claim

Figure 7.12: Seller Dashboard Interface

53

7.4.3 Administrator Interfaces

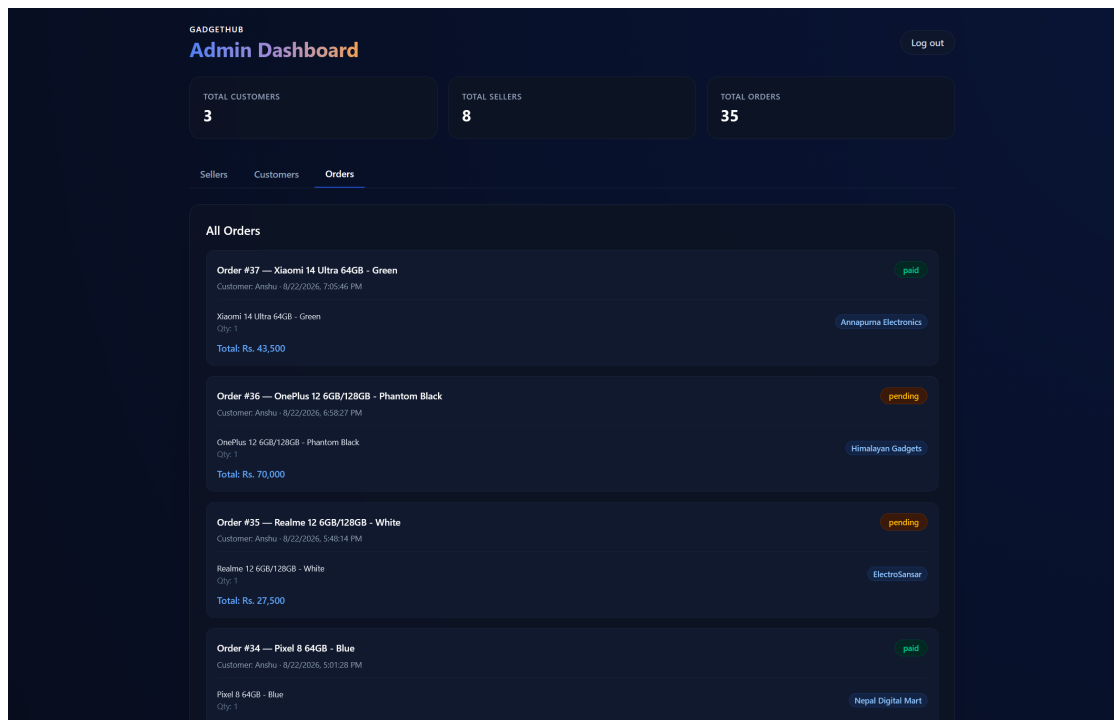


Figure 7.13: Order Management Dashboard (Paid/Unpaid)